

Time Synchronization Over Ethernet

Technical Reference

Version 1.0.0

Authors	Mehrdad Ahadi
Status	Released

Document Information

Reference Documents

No.	Source	Title	Version
[1]	AUTOS AR	Time Synchronization Protocol Specification	R19-11
[2]	AUTOS AR	Specification of Time Synchronization over Ethernet	4.3.1
[3]	AUTOS AR	Specification of Synchronized Time-Base Manager	4.4.0
[4]	IEEE	IEEE Std 802.1AS™	2011
[5]	IEEE	IEEE Std 1588™-2008	2008

Contents

1	Introduction	6
2	Functional Background	8
2.1	Synchronized Time-Base Manager (StbM).....	8
2.2	Time Synchronization Over Ethernet (EthTSyn)	13
2.2.1	Message Types	13
2.2.1.1	Event Messages Class	14
2.2.1.2	General Messages Class	14
2.2.2	Message Format	15
2.2.2.1	Common Header	16
2.2.2.2	Body	17
2.2.3	Delay Measurement in Time-Aware Gateways.....	19
2.2.3.1	Time Aware Bridge with GTM as Management CPU.....	19
2.2.3.2	Time Aware Bridge with GTM not as Management CPU	21
3	Configuration.....	24
3.1	Use case 1 - ECU acting as the GTM.....	24
3.1.1	Configuration of StbM.....	24
3.1.2	Configuration of EthTSyn for a Master ECU	26
3.2	Use case 2 - ECU Acting as a Slave ECU.....	29
3.2.1	Configuration of StbM for a Slave ECU	30
3.2.2	Configuration of EthTSyn for a Slave ECU	31
3.3	Use case 3 - Bridged network - ECU Acting as the Mgmt. CPU	31
3.3.1	Configuration of StbM for the Mgmt. CPU.....	32
3.3.2	Configuration of EthTSyn for the Mgmt. CPU.....	32
3.3.2.1	General Configuration.....	33
3.3.2.2	Configuration of a Global Time Domain.....	34
3.3.2.3	Configuration of EthTSyn Ports	37
3.3.2.4	Configuration of Master Ports	38
3.3.2.5	Configuration of Slave Ports	39
3.3.2.6	Configuration of Pdelay for Master/Slave Ports	40
4	Example Diagram.....	42
4.1	Snapshot 1.....	44
4.2	Snapshot 2.....	45
5	Glossary and Abbreviations	46
6	Contact	47

Illustrations

Figure 1-1	Timesync Modules in the AUTOSAR Layered Architecture [2].....	6
Figure 2-1	Synchronized Time-Base Manager as a Broker [3]	8
Figure 2-2	Abstract Working Principle of the Synchronized Time-Base Manager [3]	9
Figure 2-3	Interpolating the Global Time by StbM.	9
Figure 2-4	Time Correction Hierarchy [3]	10
Figure 2-5	Rate Correction [3]	11
Figure 2-6	Offset Jump Correction [3].	11
Figure 2-7	Offset Rate Adaption [3]	12
Figure 2-8	Interaction of EthTSyn to its Adjacent Modules	13
Figure 2-9	Timestamping Sequence and Distribution of the Time-Base	15
Figure 2-11	2-12 Sync Message Fields [4]	17
Figure 2-13	Follow_Up Message Fields [4]	18
Figure 2-14	Pdelay_Req Message Fields [4]	18
Figure 2-15	Pdelay_Resp Message Fields [4]	18
Figure 2-16	Pdelay_Resp_Follow_Up Message Fields [4]	19
Figure 2-17	Sync/Follow_Up Message Flow with Timestamping Points for Sync for Time Aware Bridge with GTM as Management CPU [1].....	20
Figure 2-18	Pdelay Message Flow with Timestamping Points for Time Aware Bridge with GTM as Management CPU [1]	20
Figure 2-19	Timestamping sequence for Time Aware Bridge with GTM as Management CPU	21
Figure 2-20	Sync/Follow_Up Message Flow with Timestamping points for Sync for Time Aware Bridge with GTM not as Management CPU [1]	22
Figure 2-21	Pdelay Message flow with Timestamping points for Time Aware Bridge with GTM not as Management CPU [1].....	22
Figure 2-22	Timestamping Sequence for Time Aware Bridge with GTM not as Management CPU [1]	23
Figure 3-1	Example Configuration 1 - ECU as the GTM.....	24
Figure 3-2	Sample Configuration of StbM General Container for a Master ECU	25
Figure 3-3	Configuration of StbMSynchronizedTimeBase Container for a Master ECU.....	25
Figure 3-4	Configuration of StbMLocalTimeClock for a Master ECU.....	26
Figure 3-5	Configuration of EthTSynGeneral for a Master ECU	27
Figure 3-6	Configuration of EthTSynGlobalTimeDomain	27
Figure 3-7	Configuration of an EthTSyn Port	28
Figure 3-8	Configuration of an EthTSyn Master Port.....	28
Figure 3-9	Configuration of Propagation Delay for a Master Port.....	29
Figure 3-10	Example Configuration 2 - ECU as a Slave Node.....	30
Figure 3-11	Configuration of Synchronized Time-Base Container for a Slave ECU	30
Figure 3-12	Configuration of Propagation Delay for a Slave Node	31
Figure 3-13	Example Time-aware Network.....	32
Figure 3-14	General Configuration of EthTSyn for the Mgmt. CPU	33
Figure 3-15	The configuration of Global Time Domain for the Mgmt. CPU	35
Figure 3-16	Correlation of EthTSyn, EthIf, and the Switch in a Bridged Network [2]	36
Figure 3-17	Sample Configuration of EthTSynPortConfig Container for Mgmt. CPU	37
Figure 3-18	Configuration of the Management Port	37
Figure 3-19	Sample Configuration of Global Time Master Ports in Bridged Mode.....	38
Figure 3-20	Sample Configuration of the Slave Ports for Mgmt. CPU	39
Figure 3-21	Sample Configuration of Pdelay for a Master Port.....	40
Figure 3-22	Sample Configuration of Pdelay for a Slave Port.....	40
Figure 4-1	Collaboration of EthTSyn and StbM for Distribution and Provision of the Global Time.....	43

Figure 4-2 Snapshot of StbM and EthTSyn Collaboration in the GTM..... 44

Figure 4-3 Snapshot of StbM and EthTSyn Collaboration in the Slave Node..... 45

1 Introduction

The Time synchronization Protocol is responsible for handling of the distribution of time over a communication bus. This Ethernet mechanism is based on existing PTP (Precision Time Protocol) mechanisms that are described in standards like IEEE1588 (IEEE Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems) and IEEE802.1AS (Timing and Synchronization for Time-Sensitive Applications in Bridged Local Area Networks). IEEE802.1AS, also known as gPTP (generalized Precision Time Protocol), be a profile (or subset) of IEEE1588. However, neither IEEE1588 nor IEEE802.1AS have been developed considering automotive requirements¹. Therefore, the Time Synchronization over Ethernet uses the current mechanisms as defined in IEEE802.1AS with specific extensions and/or restrictions [1] .

In AUTOSAR, the functionality of synchronization and distribution of time is handled by EthTSyn module. Considering AUTOSAR layered architecture, this module is located on top of EthIf finding StbM² on top of itself. EthTSyn and StbM are tightly coupled for providing synchronized time to Time-Sensitive automotive applications. EthTSyn cyclically provides the network time (known as the Global Time) and StbM is responsible for interpolating the local instance of the Global Time between cycles and serve the Global Time to its customers. StbM also provides the service interface for Time Synchronization to the application layer. Figure 1-1 shows the Time Synchronization related modules in the AUTOSAR Layered Architecture [2].

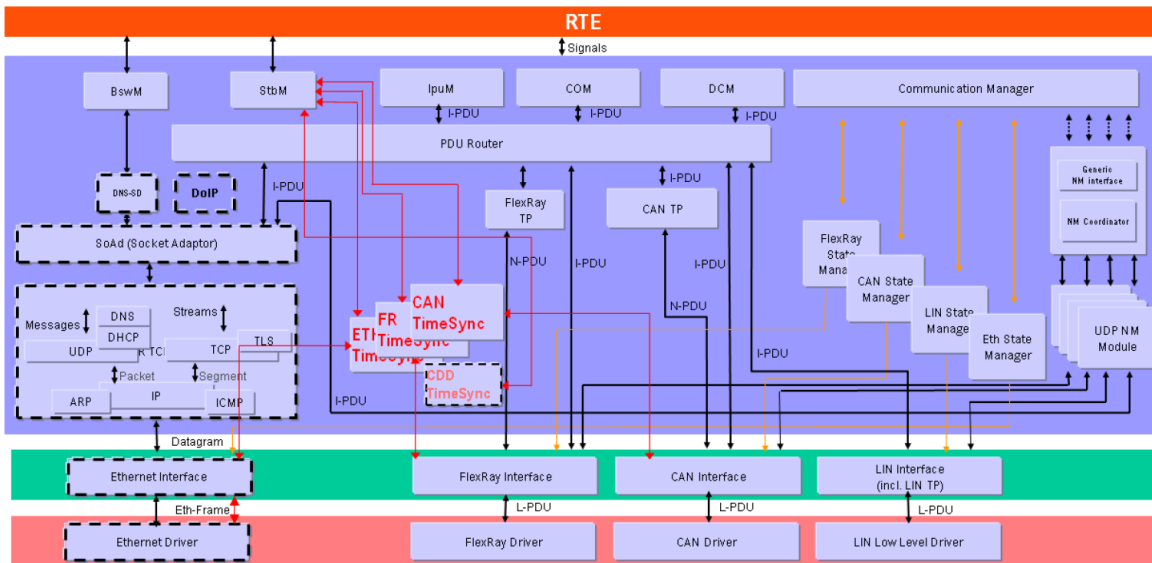


Figure 1-1 Timesync Modules in the AUTOSAR Layered Architecture [2]

As can be seen in the Figure, the communication path diverges and converges on the Ethernet Interface and does not go through the well-known Tcplp stack and SoAd³. This

> ¹ Role and functions of ECUs is known and defined a priori

> The network is static, i.e. components like ECUs, switches and characteristics like cable length, don't change during operation or even after switching off and switching on the vehicle [1].

² Synchronized time base Manager

³ Socket Adapter

means that the time synchronization protocol is Ethernet frame based¹ and network segmentation through IP layer is not considered. However, unlike IEEE 802.1AS, network segmentation is possible at the link layer with limitations². Therefore, PTP frames can have VLAN Tag [2]. Further technical details will be discussed in the next chapters.

Nodes of a time-aware network are identified by their role being a Master or a Slave. These roles and functions of ECUs are known and defined a priori. Each Time Domain only has one Global Master node known as GTM³. Although the role of a GTM can be delegated to other nodes of the network, this happens only in case of failure in the GTM and is given back after the failure is resolved. Therefore, dynamic mechanisms like determining the Global Time Master (denoted as grandmaster in IEEE802.1AS) by the best master clock algorithm (BMCA) during operation are not required [1].

As previously mentioned, EthTSyn and StbM are functionally tightly coupled. Due to this, in this document, significant functional points of these modules are discussed, and necessary configuration hints will be explained. The configuration examples will consider an abstraction of typical time-aware network use-cases and are demonstrated in Vector's DaVinci Configurator tool. The rest of the document is organized as follows:

Section 2 briefly describes functional overview of two modules EthTSyn and StbM as well as their practical semantics applicable to AUTOSAR and Automotive Embedded Systems. Section 3 presents the associated configuration parameters as well as the relevant numerical values for our example scenario. Finally, section 4 will present an example diagram depicting the functional concept utterly.

¹ Commonly known as PTP frames

² For further information, please refer to see [2] .

³ Global Time Master

2 Functional Background

In this chapter, a preliminary foundation of Time Synchronization Over Ethernet will be reviewed. As discussed in the previous section, in automotive use-cases, as specified by AUTOSAR, synchronization of time is accomplished via two main functional roles, Global time provider and synchronized time broker. These roles are assigned to EthTSyn and StbM modules respectively. Accordingly, in the following subsections, the main tasks of these two modules are to be introduced.

2.1 Synchronized Time-Base Manager (StbM)

Figure 1 illustrates how the Synchronized Time-Base Manager interacts with other modules.

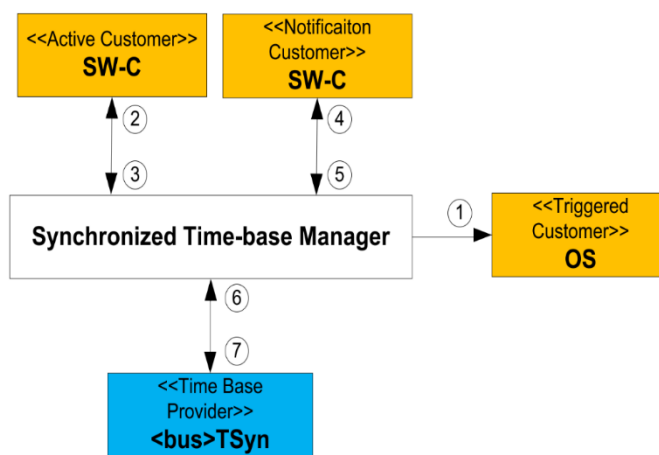


Figure 2-1 Synchronized Time-Base Manager as a Broker [3].

As seen in the figure, StbM is bus independent. The Synchronized Time-Base Manager itself does not provide any time agreement protocols to synchronize its local Time-Bases to Time-Bases on other nodes. It interacts with the <bus>TSyn modules of the BSW to achieve such synchronization. With the information retrieved from the provider modules, the Synchronized Time-Base Manager can synchronize its Time-Bases to Time-Bases on other nodes [3].

StbM maintains a local instance of the Global time and serves it to its so-called customers. Three types of customers may be distinguished: Triggered customer, Active customer, and Notification customer (for further information about the details, please refer to [3]). Thus, the StbM acts as Time-Base broker by offering the customers access to the Synchronized Time-Bases.

In general, distribution of time is cyclic (e.g. every 125ms) by EthTSyn. Due to this, StbM must provide access to the Global Time during the time between two consecutive updates of the time. StbM provides the time to its customers through interpolating the local instance of the Global Time whenever an access is required. This is realized by using a hardware reference clock as well as a software counter. This combination forms Virtual Local Time. The schematic of maintaining the local instance of the global time is illustrated in the following figure [3].

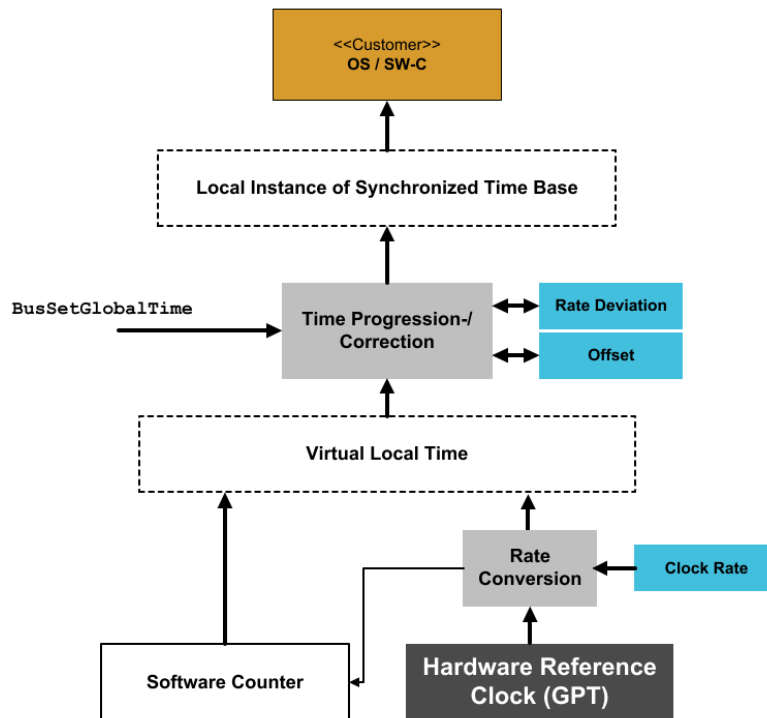


Figure 2-2 Abstract Working Principle of the Synchronized Time-Base Manager [3] .

The Figure, which abstracts the working principle of StbM, can be best explained via an example and introducing some important terms which will further reveal the interaction of different components in StbM when it comes to, for example, reading the current Global Time. Reading the current time can be done via calling `StbM_GetCurrentTime()` API and returns the local instance of the Global Time in standard format. Please note that this API can be called at any point in time but the GTM updates the Global Time every e.g. 125ms. Therefore, StbM shall properly calculate the requested Global Time in the intermediate spots (please see the following figure).

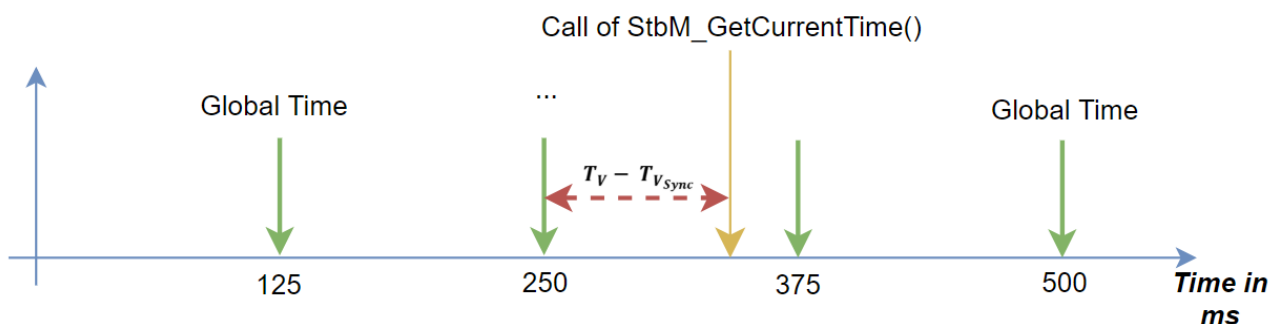


Figure 2-3 Interpolating the Global Time by StbM.

The following shows how different components of the Figure incorporate to form the functionality of StbM:

- > **Virtual Local Time:** is a time used for driving the Local Instance of the Global Time and progresses monotonously by a local hardware reference (OS counter, GPT counter, or Ethernet free running counter).

- > $T_{L_{Sync}}$: Current value of the local instance of the Global time before newly received Global Time is applied via `StbM_BusSetGlobalTime` (`StbM_BusSetGlobalTime` forwards the received Global Time from `EthTSyn` to `StbM`)
- > T_G : Received value of the Global Time (Newly received Time-Base value from `EthTSyn`)
- > T_V : Current value of the Virtual Local Time
- > $T_{V_{Sync}}$: Snapshot of the Virtual Local Time when a new Global Time is received
- > $T_{G_{Sync}}$: Value of the local instance of the Global Time at the synchronization event (the moment the Global Time was received via `EthTSyn`)

Get Current Time:

1. **Get T_V :** Virtual Time will be updated with adding the elapsed time of the hardware reference clock.
2. Get $T_{V_{Sync}}$
3. Get $T_{G_{Sync}}$

Current Time: $T_{G_{Sync}} + (T_V - T_{V_{Sync}})$

$T_V - T_{V_{Sync}}$ simply is a measurement preserving the amount of time which has passed from the last received Global Time (Interpolating the Global Time by `StbM`). Since it is maintained completely locally, it can be influenced by the clock drifts of the local hardware. Furthermore, when a new Global Time is received from the bus, there might be an offset between the local instance of the Global Time and the received Global Time. Therefore, it might be needed to perform a kind of time correction defined as seen in the following Figure:

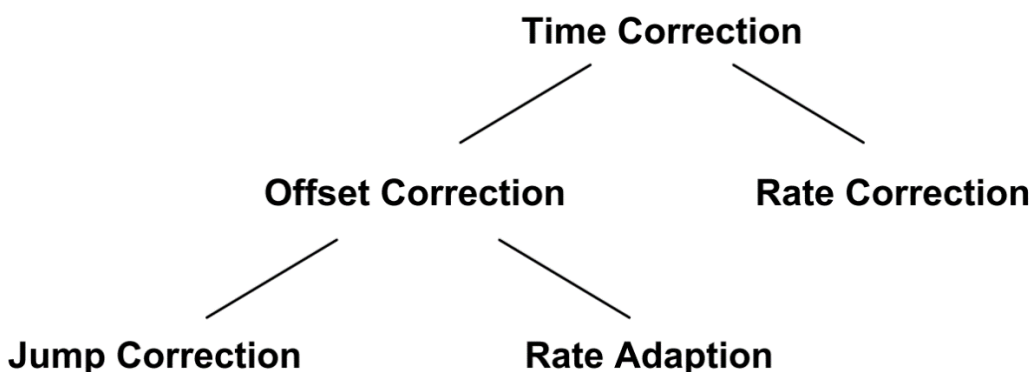


Figure 2-4 Time Correction Hierarchy [3] .

- > **Rate Correction:** Rate Correction corrects the rate deviation of the local hardware reference clock. It determines the correction factor in the scope of a measurement. This correction factor is however not fixed but updated after each successful

measurement. The working principle of Rate Correction is not to adjust the local hardware reference clock in order to let it progress with the correct rate. Instead Rate Correction only corrects the values of the local instance of the Time-Base on-the-fly when they are read: $r * (T_V - T_{V_{\text{sync}}})$ where r is the measured correction rate [3].

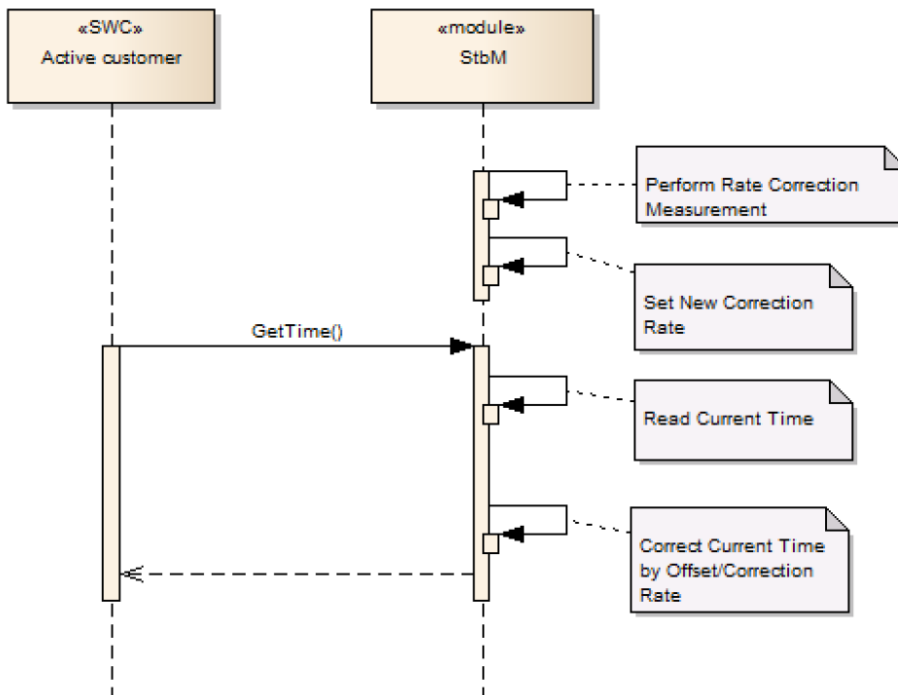


Figure 2-5 Rate Correction [3]

- > **Offset Correction/Jump Correction:** Jump Correction corrects absolute time offsets in a single step by adding the offset to the local instance of the Global Time (which is equivalent to taking over the value of the Global Time received from EthTSyn) [3].

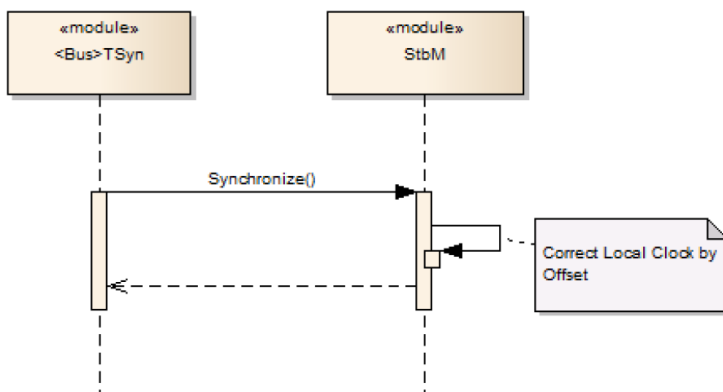


Figure 2-6 Offset Jump Correction [3].

- > **Offset Correction/Rate Adaptation:** Rate Adaption corrects time offsets gradually within a predefined timespan. Hereto, Rate Adaption switches the rate of the local

instance of the Time-Base temporarily to a different value. This rate is chosen to completely eliminate the offset within the preconfigured timespan. Like Rate Correction, Rate Adaption does not adjust the local instance of the Time-Base (including hardware reference clock). It merely corrects the clock values on-the-fly when they are read [3].

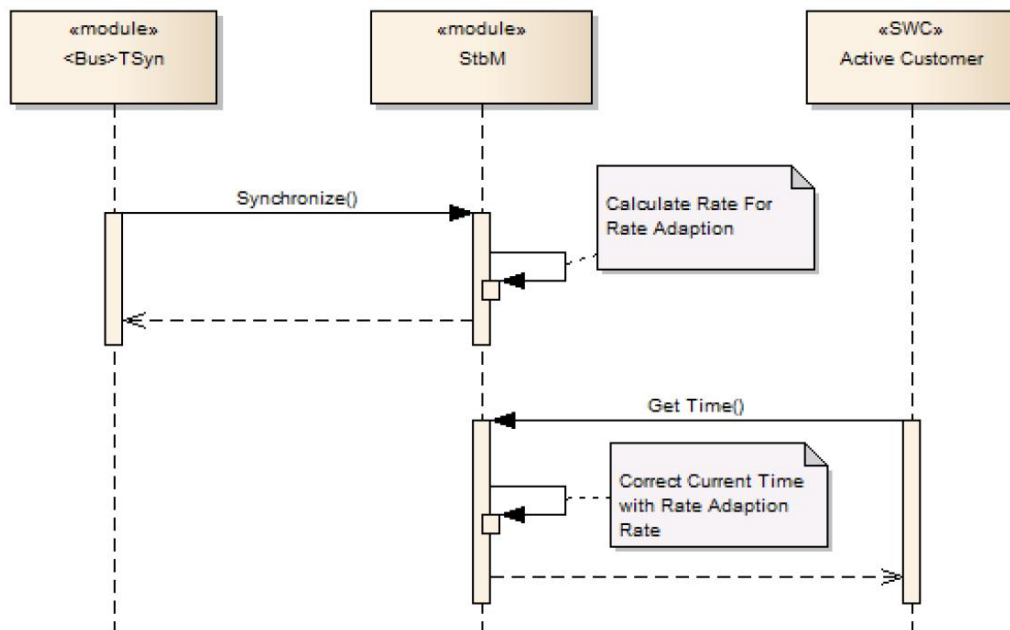


Figure 2-7 Offset Rate Adaption [3].

Looking back to the rate correction, StbM applies an approach to detect and eliminate rate deviations of local hardware clock. StbM determines the rate deviation in the scope of a measurement. In fact, the algorithm is relatively simple. StbM starts measuring the rate deviation on reception of the Global Time from EthTSyn (in the scope of StbM_BusSetGlobalTime()). Then it stores the following elements:

- > T_{GStart} – Received Global Time
- > T_{VStart} – Virtual Local Time

At the end of the rate measurement duration, the following elements are additionally stored in the scope of StbM_BusSetGlobalTime():

- > T_{GStop} – Received Global Time
- > T_{VStop} – Virtual Local Time

$T_{GStop} - T_{GStart}$ demonstrates the time progression in the Time Master and $T_{VStop} - T_{VStart}$ shows the time progression in the Time Slave. Therefore, rate correction factor is calculated as: $r = (T_{GStop} - T_{GStart}) / (T_{VStop} - T_{VStart})$ [3].

The above shortly introduced some main aspects of StbM module from functional point of view. Of course, there are more to what was presented here. Information such as time

recording, synchronization states, Offset Time-Bases, etc. where intentionally skipped. However, for further information about all the details of the Synchronized Time-Base Manager, I would suggest referring to [3]. In the next section, we will have a survey of the essential syntax and semantics of EthTSyn module.

2.2 Time Synchronization Over Ethernet (EthTSyn)

As discussed in the previous section, EthTSyn is responsible for distributing the Global Time over an Ethernet bus. In AUTOSAR, this module is located between StbM and EthIf as shown in the following figure.

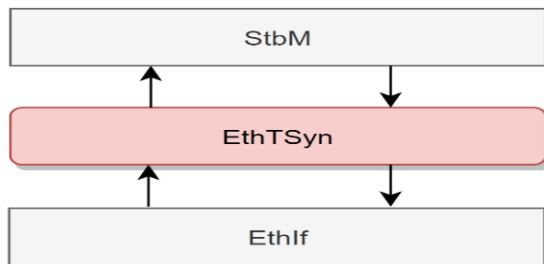


Figure 2-8 Interaction of EthTSyn to its Adjacent Modules

Like StbM, there are two roles associated to the nodes of a time-aware network, Master and Slave:

Master Node: A Time Master is an entity which is the master for a certain Time-Base and which propagates the Global Time to a set of Time Slaves within a certain segment of a communication network, being a source for this Time-Base. If a Time Master is also the owner of the Time-Base, then it is the Global Time Master (GTM) [1].

Slave Node: A Time Slave is an entity, which is the recipient for a certain Time-Base within a certain segment of a communication network, being a consumer for this Time-Base.

Gateway Node: A Time Gateway typically consists of one Time Slave and one or more Time Masters. When mapping time entities to real ECUs, an ECU could be Time Master (or even Global Time Master) for one Time-Base and Time Slave for another Time-Base.

Unless otherwise specified, the Ethernet frames carrying time synchronization information are called PTP frames and have their own ethernet type 0x88F7. As well, the communication is one to many (multicast). This means the PTP frames are multicast over the network and have a specified destination MAC address of 01:80:C2:00:00:0E [4].

Like other communication protocols, e.g. Ethernet, Tcp/Ip, Can, CanTp, etc. EthTSyn also has its own message types, structure, and semantics. In the following subsections, different types of messages and their communication sequence is discussed.

2.2.1 Message Types

There are mainly five types of PTP messages in use specified in [4]¹. These messages are shown in the following figure and categorized in two classes: Event Messages and General Messages [4].

¹ For the other message types e.g. Announce message and Signaling message, please refer to [4].

Message Type	Message Class	Value
Sync	Event	0x0
Pdelay_Req	Event	0x2
Pdelay_Resp	Event	0x3
Follow_Up	General	0x8
Pdelay_Resp_Follow_Up	General	0xA

Table 2-1 PP Message Types and Values for messageType Field

2.2.1.1 Event Messages Class

From functional point of view, there is an Ingress/Egress timestamp associated to the event messages. Ingress timestamps refer to the moment the message is received from the bus and Egress timestamps are captured once the messages are transmitted to the bus.

Sync: A Sync message contains time-synchronization information that originates from the GTM. Sync messages are transmitted from a Master node and in fact do not carry any part of the Global Time. The sender of a Sync message retrieves the egress timestamp in order to calculate the transmission delay¹ and the receiver of a Sync message retrieves the ingress timestamp to measure the time difference between the reception of a Sync message and the time its associated Follow-Up message will be received in the EthTSyn layer.

Pdelay_Req: A Pdelay_Req message is used for triggering the measurement of the propagation delay typically from a slave node. The request might be issued from a Master node as well. However, at the moment, it is the slave node that uses the value of the path delay to properly calculate the Global Time on its own side. The sender of a Pdelay_Req message retrieves the egress time stamp as part of measurement process. The receiver of a Pdelay_Req message retrieves the ingress timestamp to notify when the request was received.

Pdelay_Resp: A Pdelay_Resp message is transmitted by the Master node to provide the time at which the corresponding Pdelay_Req was received. The sender of a Pdelay_Resp message retrieves the egress timestamp in order to help calculating the turnaround time. The receiver of a Pdelay_Resp message retrieves the ingress timestamp so that the Slave node can measure its own turnaround time.

2.2.1.2 General Messages Class

General messages are not required to be timestamped.

Follow_Up: A Follow_Up message as its name suggests follows a Sync message and carries the Global Time as well as any correction time which is required to compensate any transmission delays. Follow_Up messages are also transmitted from a Master node.

Pdelay_Resp_Follow_Up: A Pdelay_Resp_Follow_Up message carries the egress timestamp of the related Pdelay_Resp message.

The timestamping sequence as well as the mentioned measurements are demonstrated by the following figure.

¹ Transmission delay: the time between request of the transmission in the upper layers and the moment transmission was finalized in the hardware.

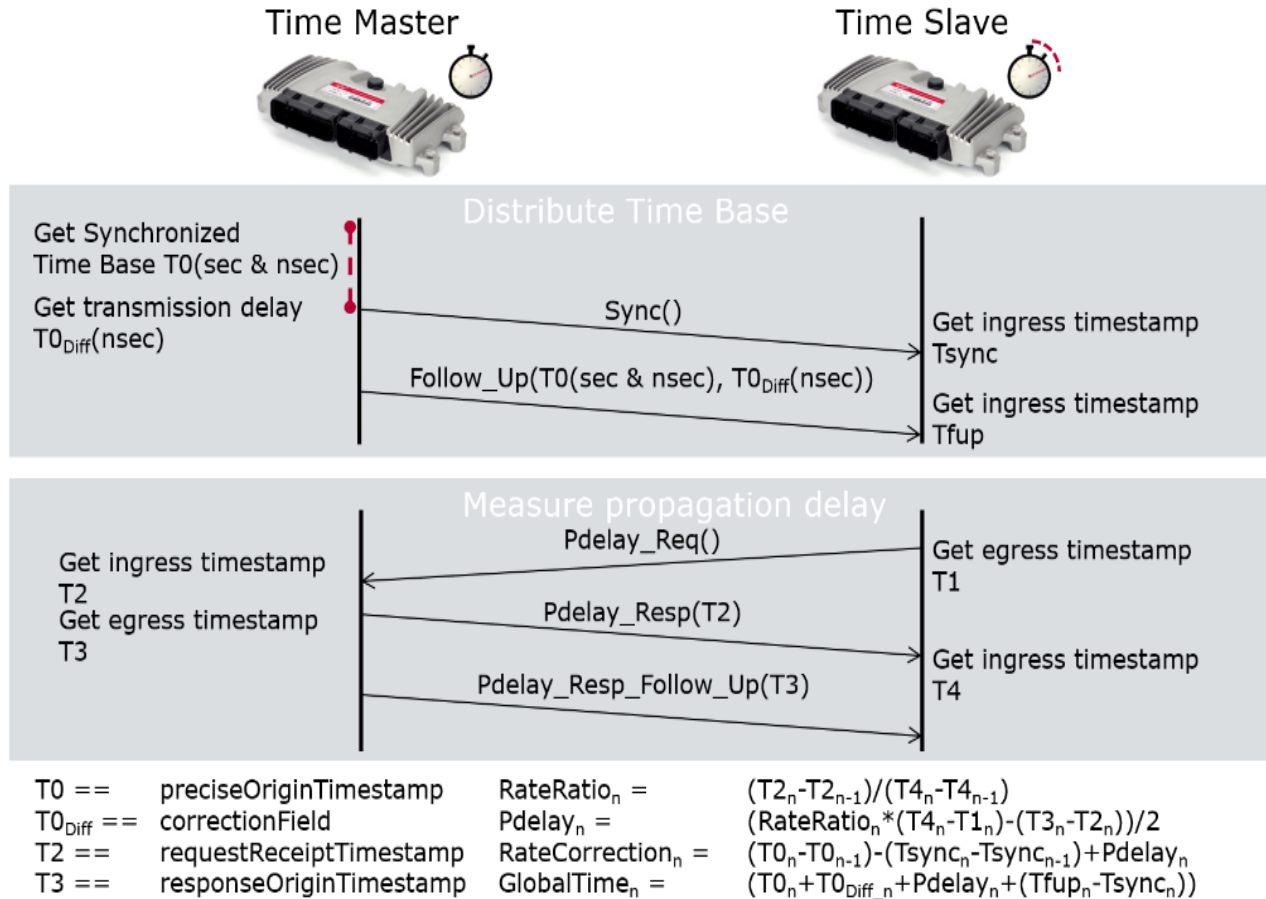


Figure 2-9 Timestamping Sequence and Distribution of the Time-Base

As can be seen in the figure, the GTM gets the Global Time from StbM (T_0). Once the Sync message is transmitted, the egress timestamp is captured and subtracted by T_0 to calculate the transmission delay of the Sync message (T_{0diff}). Later in time, a Follow_Up message is transmitted to publish the Global Time as well as the transmission delay. On the other side, once the Sync message is arrived, the Slave node gets the ingress timestamp (T_{sync}) also ingress timestamp of the Follow_Up message (T_{Fup}). $T_{Fup} - T_{sync}$ is the time passed after the Global Time was read in the Master Node. However, as seen in the figure, it is not all the time portion which covers the same time progress in the Master Node. The missing parts are the transmission delay which is already transmitted by the Follow_Up message and the path delay.

To measure the propagation/path delay, the mechanism is reasonably straightforward. The Slave Node measures the time spent from the request of the Pdelay till it receives the response from the Master Node ($T_4 - T_1$). The Master node, on the other hand, notifies how long of this time belongs to the processing time in the Master node ($T_3 - T_2$). Then, the Slave Node subtracts these time portions to calculate the propagation delay in both directions. Finally, the result is divided by 2 to have Pdelay of each direction.

2.2.2 Message Format

As previously discussed, Time Synchronization messages are carried over ethernet frames and called PTP frames. These PTP frames have their own ethernet type and their destination MAC address is set to a dedicated multicast address:

Destination address
01-80-C2-00-00-0E
NOTE – This address is taken form Table 8-1 of IEEE Std 802.IQ-2005

Table 2-2 Destination address for Sync, Follow_Up, Pdelay_Req, Pdelay_Resp, and Pdelay_Resp_Follow_Up messages [4]

Ethertype
88F7 ₁₆

Table 2-3 Ethertype for Sync, Follow_Up, Pdelay_Req, Pdelay_Resp, and Pdelay_Resp_Follow_Up messages [4]

The content of PTP frames are composed of a common header followed by a message dependent body and, if present, a suffix that contains one or more TLVs (for more information about TLVs, please refer to [4]).

2.2.2.1 Common Header

The Common Header is 34 bytes long and common to all PTP messages. All reserved bits are set to 0 and ignored by the receiver¹. The following figure shows the fields of the header:

transportSpecific	messageType
Reserved	versionPTP
messageLength	
domainNumber	
reserved	
flags	
correctionField	
reserved	
sourcePortIdentity	
sequenceId	
control	
logMessageInterval	

Table 2-4 PTP Common Header Fields [4]

> **Message Type:** This field carries the message type and shall be one of the following:

Message Type	Message Class	Value
Sync	Event	0x0
Pdelay_Req	Event	0x2
Pdelay_Resp	Event	0x3
Follow_Up	General	0x8
Pdelay_Resp_Follow_Up	General	0xA

¹ Unless otherwise said

Table 2-5 Values for messageType field [4]

- > **TransportSpecific:** The value is 0x1.
- > **VersionPTP:** The value is 0x2 and it indicates the version number of IEEE 1588 PTP used in the PTP profile contained in the standard.
- > **MessageLength:** This field indicates the full length of the PTP message including the header body as well as any carried suffixes.
- > **DomainNumber:** This field identified the clock domain the carrying PTP message belongs to. A clock domain, as mentioned before, is a logical grouping of clocks that synchronize to each other using the protocol.
- > **Reserved:** All zero.
- > **Flags:** This field contains various flags to indicate the status.
- > **CorrectionField:** This field carries the correction value in nanoseconds. In switched networks, this field is populated with the switch residence time and added to the current value in the next hops of the journey when forwarded. The value of this field is multiplied by 2^{16} . For example, 2.5 ns is represented as 0x00000000000028000.
- > **Reserved:** All zero.
- > **SourcePortIdentity:** This field indicates the originating port for this message.
- > **SequenceId:** This is the sequence Id for the current message.
- > **ControlField:** This is a historical field whose value depends on the message type. Similar to the MessageType field but with less options.
- > **LogMessageInterval:** This field is determined by the type of the message. For further information about this field, please refer to [4].

2.2.2.2 Body

This part of a PTP message is type dependent and the layout is specified as follows:

Sync Message:

Bits								Octets	Offset
8	7	6	5	4	3	2	1		
Common Header								34	0
reserved								10	34

Figure 2-10 2-11 Sync Message Fields [4]

Follow-Up Message:

Bits								Octets	Offset
8	7	6	5	4	3	2	1		
Common Header								34	0
preciseOriginTimestamp								10	34
Follow_Up information TLV								32	44

Figure 2-12 Follow_Up Message Fields [4]

preciseOriginTimestamp: The value of the preciseOriginTimestamp field is the sourceTime of the GTM which was recorded at the moment when issuing the transmission of Sync message. Remember that this time plus the time carried by the correctionField in the corresponding Common Header compose the synchronized time. Later, adding sum of these values to the propagation delay of the link forms the Global Time in slave nodes.

Follow_Up information TLV: For further information about this struct, please refer to [4].

Pdelay Request Message:

Bits								Octets	Offset
8	7	6	5	4	3	2	1		
Common Header								34	0
reserved								10	34
reserved								10	44

Figure 2-13 Pdelay_Req Message Fields [4]

Pdelay Response Message:

Bits								Octets	Offset
8	7	6	5	4	3	2	1		
Common Header								34	0
requestReceiptTimestamp								10	34
requestingPortIdentity								10	44

Figure 2-14 Pdelay_Resp Message Fields [4]

requestReceiptTimestamp: This field carries the ingress timestamp of the associated Pdelay_Req message.

requestingPortIdentity: The value is the value of the sourcePortIdentity field of the associated Pdelay_Req message.

Pdelay Response Follow_Up Message:

Bits								Octets	Offset
8	7	6	5	4	3	2	1		
Common Header								34	0
responseOriginTimestamp								10	34
requestingPortIdentity								10	44

Figure 2-15 Pdelay_Resp_Follow_Up Message Fields [4]

responseOriginTimestamp: This field carries the egress timestamp of the associated Pdelay_Resp message.

requestingPortIdentity: The value is the value of the sourcePortIdentity field of the associated Pdelay_Req message.

In the clauses above, the main PTP frames as well as their message layout were shortly introduced. For further information about the details of the messages e.g. announce messages, TLV fields, etc. please refer to the standard [4]. In the following subsection, delay measurement in switched networks is discussed in two different common setups.

2.2.3 Delay Measurement in Time-Aware Gateways

Time Synchronization over switched networks, compared to direct link Master/Slave connections, involve switches in the way of PTP frames. These switches, depending on their setup, add some delays of processing which need to compensate. Measuring this kind of delay which is mainly because of processing overhead, queuing under load, etc. inside the switch, is examined in two different use cases:

2.2.3.1 Time Aware Bridge with GTM as Management CPU

In the time-aware networks, MCU is connected to the switch via a host¹ port (typically xMii connection) and manages the switch and is responsible for handling of PTP traffic. In this setup, GTM is the management CPU (calling it Mgmt. CPU). This means, the PTP traffic are distributed to the network but not forwarded. Therefore, the lag caused by switch has only one way seen in the transmission path.

¹ The host port is a special port of the switch which might be responsible for some ethernet traffic.

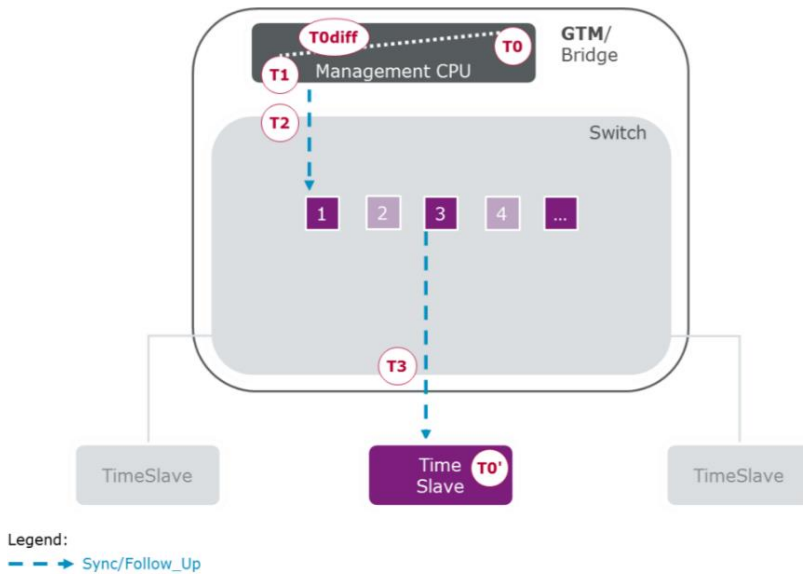


Figure 2-16 Sync/Follow_Up Message Flow with Timestamping Points for Sync for Time Aware Bridge with GTM as Management CPU [1]

As seen in the figure, during the flow of PTP messages, the time between T_2 and T_3 is the delay incurred by the switch and shall be compensated. This is called switch Tx residence time. Similarly, the flow of Pdelay messages is affected as follows:

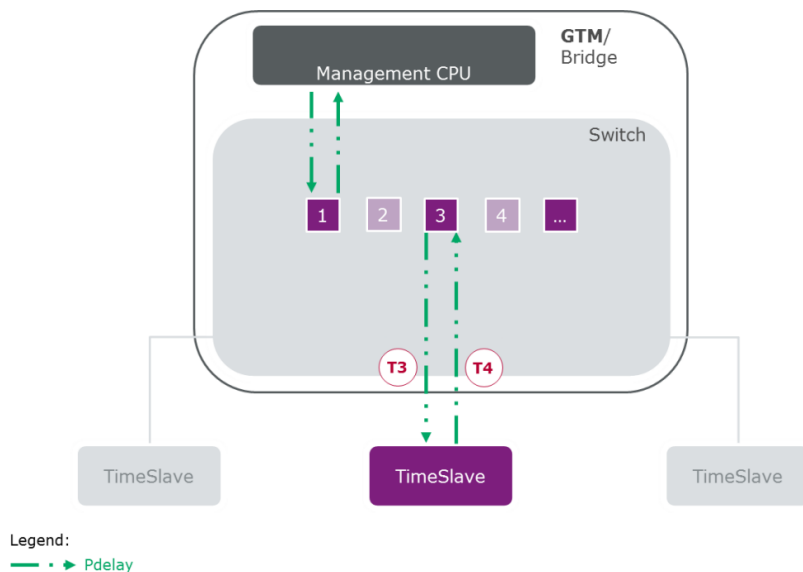


Figure 2-17 Pdelay Message Flow with Timestamping Points for Time Aware Bridge with GTM as Management CPU [1]

The following figure shows in which sequence timestamps are retrieved and how the Global Time is adapted accordingly.

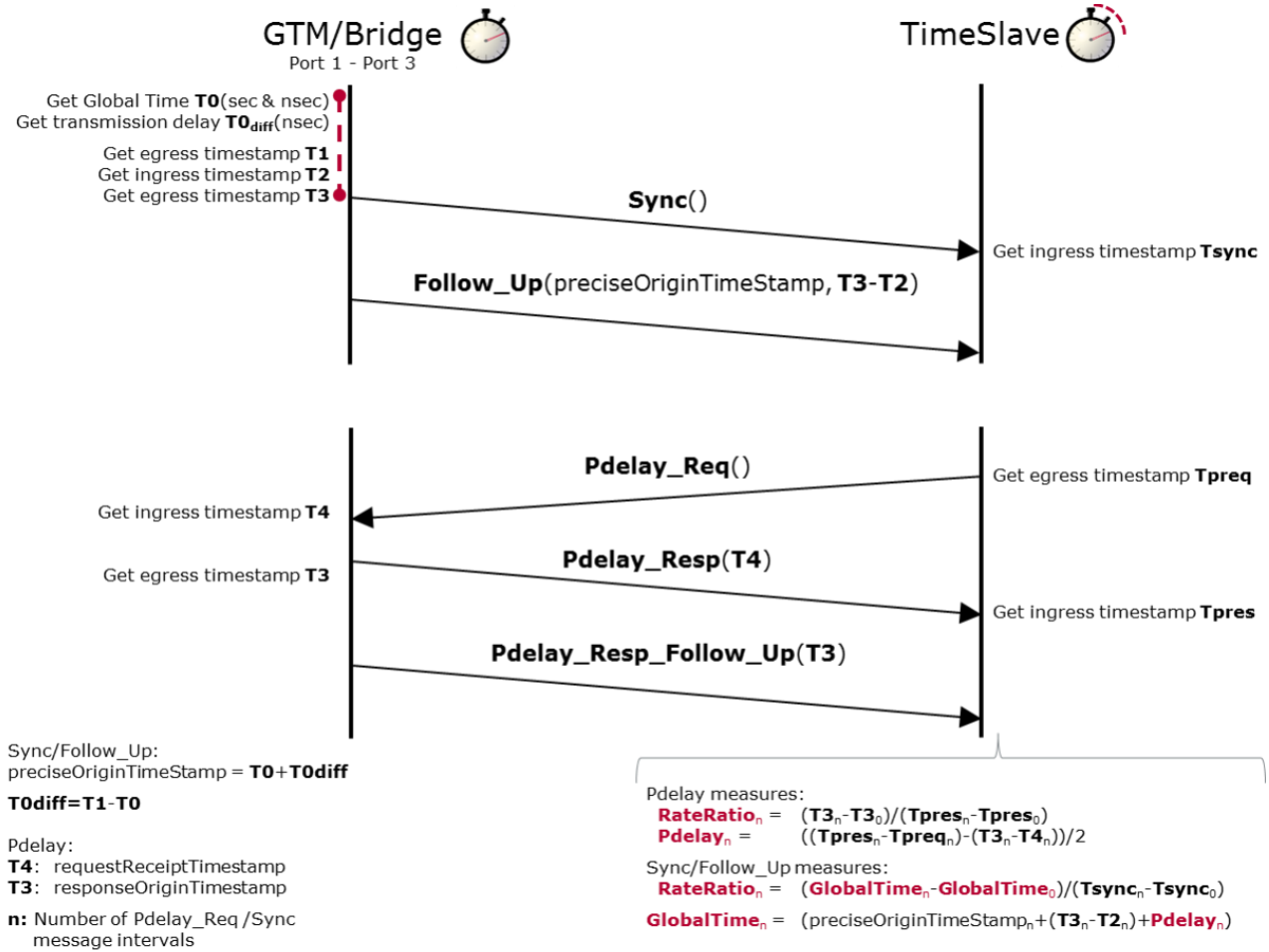


Figure 2-18 Timestamping sequence for Time Aware Bridge with GTM as Management CPU

The switch ports also shall provide timestamping so that the switch residence time is measured accurately and added to the transmission delay. The adapted transmission delay is added to the correctionField of PTP Follow_Up messages and transmitted to the next hop. Please note that for Pdelay measurement, the switch residence time is implicitly carried by Pdelay_Resp and Pdelay_Resp_Follow_Up messages.

2.2.3.2 Time Aware Bridge with GTM not as Management CPU

In this setup, the GTM is a connected node to the switch and the Mgmt. CPU is responsible for forwarding the PTP traffic. In this scenario, PTP frames are received from a switch port going through switch Rx residence time, being processed by the Mgmt. CPU, and forwarded to the rest of the network (Tx switch residence time). Therefore, all these influencing divisions should be considered when constructing the correctionField of PTP Follow_Up messages.

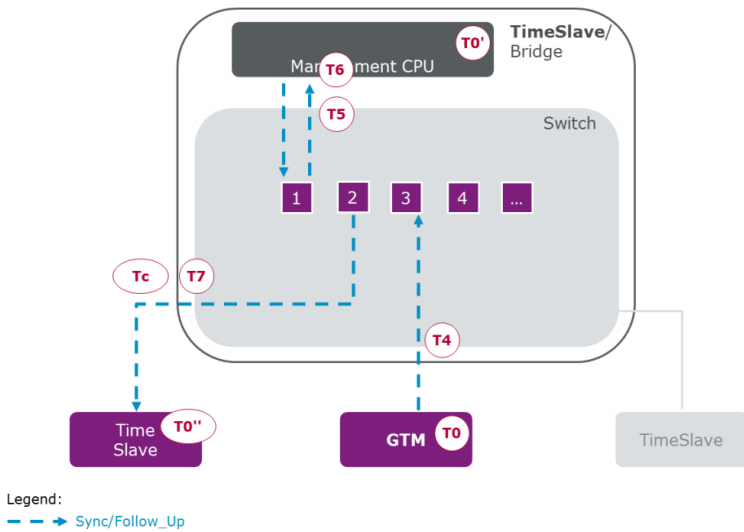


Figure 2-19 Sync/Follow_Up Message Flow with Timestamping points for Sync for Time Aware Bridge with GTM not as Management CPU [1]

In the figure above, from T_4 to T_7 is the switch residence time as well as the transmission delay of the Mgmt. CPU (T_4 to T_5 as Rx residence time, T_6 to T_7 as Tx residence time, the rest is the delay of Mgmt. CPU). Therefore, being able to retrieve the ingress timestamp of port 3 when a Sync message is received, and egress timestamp of port 2 when the Sync message is forwarded, the Mgmt. CPU can properly adapt the correctionField of the upcoming Follow_Up message as shown in Figure 2-21. Different from the previous case, in this scenario, propagation delay of the link to the GTM (link to port 3 in the following figure) shall also be added to the correctionField.

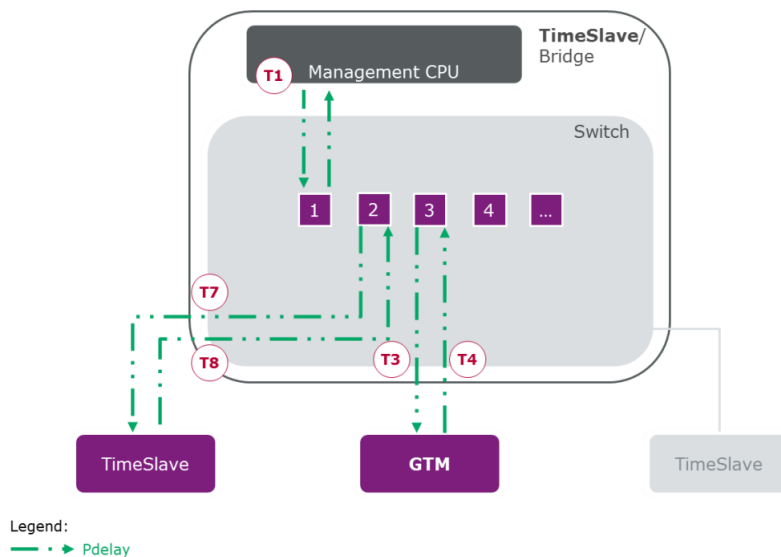


Figure 2-20 Pdelay Message flow with Timestamping points for Time Aware Bridge with GTM not as Management CPU [1]

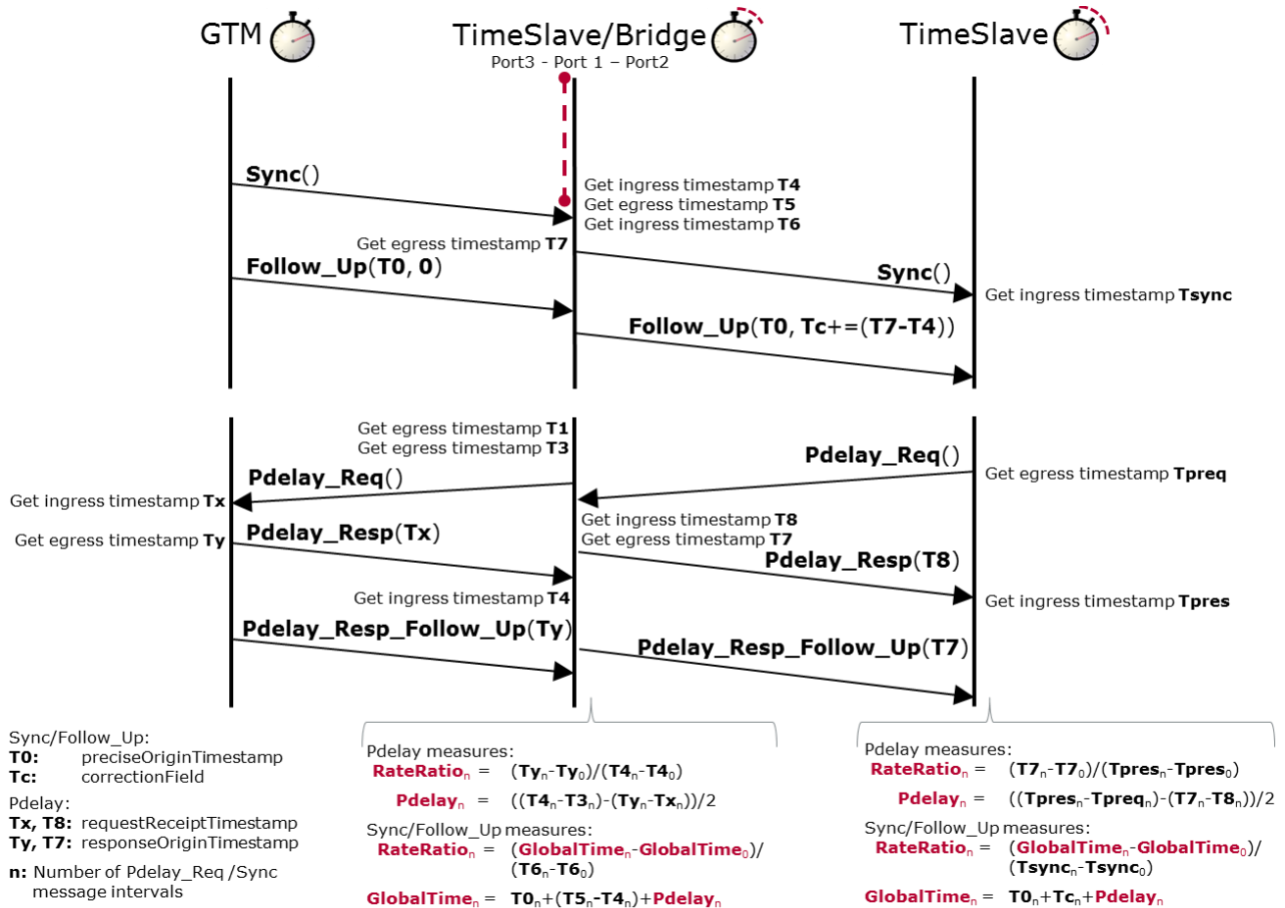


Figure 2-21 Timestamping Sequence for Time Aware Bridge with GTM not as Management CPU [1]

In this section, general functionality of time distribution over ethernet was briefly discussed and it was shown that how different PTP frames and their fields distribute the time over an Ethernet bus. As well, the importance of timestamp functionality was depicted. In the following section, some common time synchronization scenarios¹ will be reviewed and the important points for a proper configuration will be exemplified using Vector DaVinci Configurator Tool.

¹ With respect to the network topology and node roles

3 Configuration

In this section, some conventional setups are to be illustrated and their example configuration will be demonstrated. Since the roles of StbM and EthTSyn are correlated, the configuration of both might be necessary to shown. Therefore, for each example setup, first configuration points of StbM is shown followed by the corresponding role in EthTSyn. For all examples, we assume that both modules are integrated in the project and all dependent modules are properly configured (EthDrv, EthIf, EthSwt, ...).

Three example use cases are:

- > **Use case 1:** ECUToNode, ECU acting as the GTM (Global Time Master)
- > **Use case 2:** ECUToNode, ECU acting as a time Slave
- > **Use case 3:** Bridged network ECU acting as the Mgmt. CPU

3.1 Use case 1 - ECU acting as the GTM

In this example setup, two ECUs are connected to each other over a direct Ethernet link (Figure 3-1) and the ECU which is the GTM is the focus of the configuration. The expected behavior is transmitting the Sync/Follow_Up messages every 125ms and participating in the propagation delay measurement process.

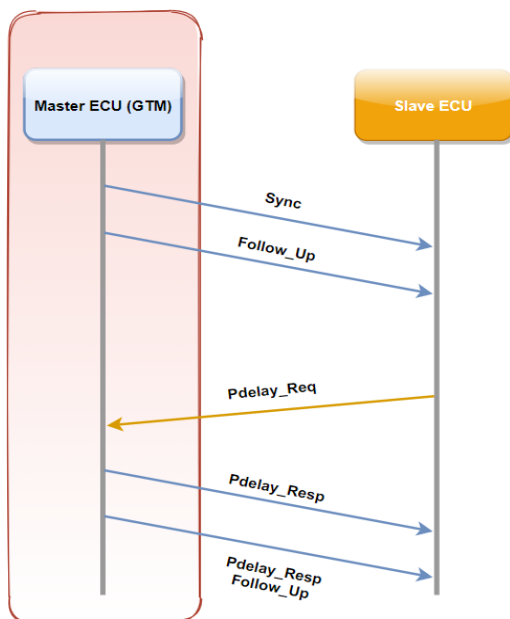
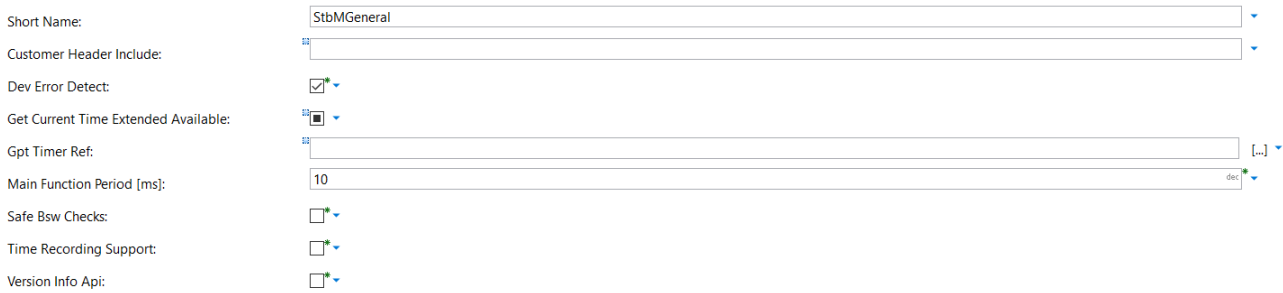


Figure 3-1 Example Configuration 1 - ECU as the GTM

Firstly, we go through the configuration of StbM and setting the relevant parameters properly for the Master role:

3.1.1 Configuration of StbM

The configuration of StbM is pretty straight forward. The General config can be adapted as Figure 3-2 shows:



Short Name: StbMGeneral

Customer Header Include:

Dev Error Detect: ☒

Get Current Time Extended Available: ☒

Gpt Timer Ref: 10

Main Function Period [ms]: 10

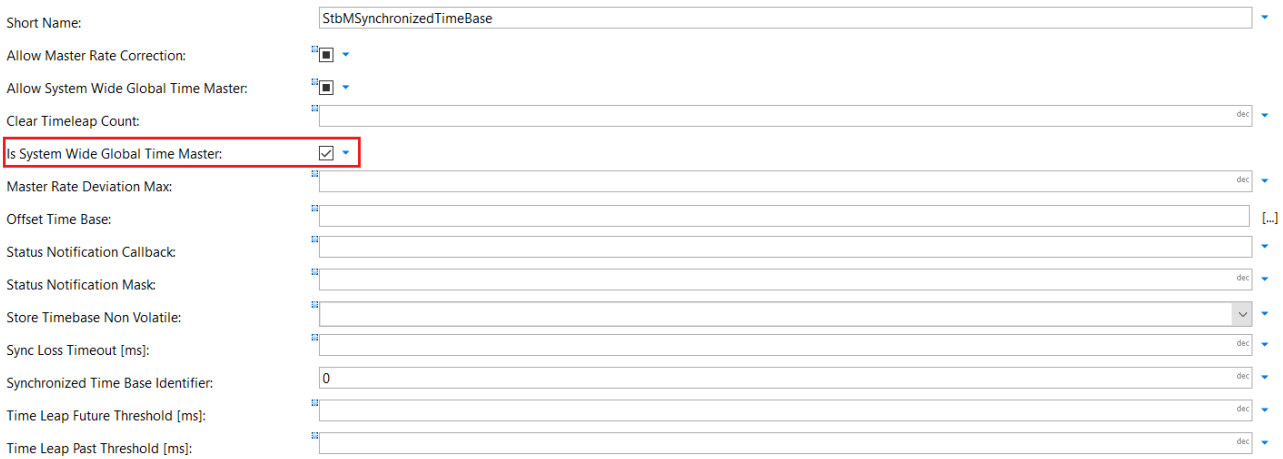
Safe Bsw Checks: ☐

Time Recording Support: ☐

Version Info Api: ☐

Figure 3-2 Sample Configuration of StbM General Container for a Master ECU

Now we should create one Synchronized Time-Base¹ and configure it as follows:



Short Name: StbMSynchronizedTimeBase

Allow Master Rate Correction: ☒

Allow System Wide Global Time Master: ☒

Clear Timeleap Count:

Is System Wide Global Time Master: ☒

Master Rate Deviation Max:

Offset Time Base:

Status Notification Callback:

Status Notification Mask:

Store Timebase Non Volatile:

Sync Loss Timeout [ms]:

Synchronized Time Base Identifier: 0

Time Leap Future Threshold [ms]:

Time Leap Past Threshold [ms]:

Figure 3-3 Configuration of StbMSynchronizedTimeBase Container for a Master ECU

- > **Is System Wide Global Time Master:** This parameter enables if this Synchronized Time-Base is acting as a Master role (checked) or a Slave role (unchecked).
- > **Synchronized Time-Base Identifier:** Identification of the Synchronized Time-Base via a unique identifier. The following ranges shall be considered:
 - > 0 ... 15: Synchronized Time-Bases
 - > 16 ... 31: Offset Time-Bases
 - > 32 ... 127: Pure Local Time-Bases
 - > 128 ... 65535: Reserved

In the next step, one container of StbMLocalTimeClock should be added for configuring the reference clock hardware used by the StbM for interpolating the Global Time (Figure 3-4)².

¹ /MICROSAR/StbM/StbMSynchronizedTimeBase

² /MICROSAR/StbM/StbMSynchronizedTimeBase/StbMLocalTimeClock

Short Name:	StbMLocalTimeClock
Clock Frequency [Hz]:	100000000 dec
Clock Prescaler:	1 dec
Local Time Hardware:	/ActiveEcuC/Gpt/GptChannelConfigSet/GptChannelConfiguration_0 [...]

Figure 3-4 Configuration of StbMLocalTimeClock for a Master ECU

- > **Clock Frequency:** The value of the clock frequency depends on the chosen Local Time Hardware. In our example, every tick of the clock is 10ns and the chosen clock has the frequency of 100MHz.
- > **LocalTimeHardware:** Our chosen reference is a Gpt clock. In general, there are three possible options:
 - > System Clock
 - > GPT Clock
 - > Ethernet Clock

For now, this configuration would be enough for the basic functionality of StbM acting as the Global Time Master. However, one more step which is required to equip the ECU is setting the Global Time in the code. In order to do so, StbM_SetGlobalTime() shall be used. The details of this API can be found in SWS_StbM_00213. The following shows an example:



Example

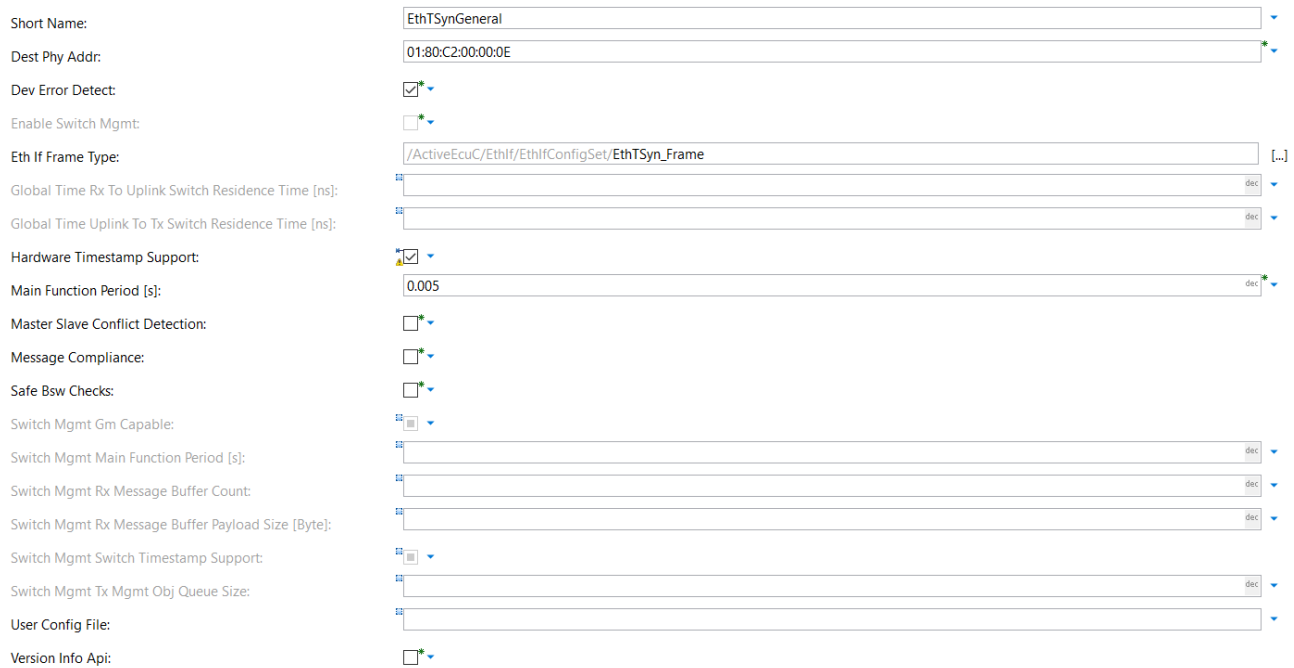
```
StbM_TimeStampType timeStamp;
StbM_UserDataType usrData;

timeStamp.nanoseconds = 0;
timeStamp.seconds = 0;
timeStamp.secondsHi = 0;
timeStamp.timeBaseStatus = 0;
usrData.userDataLength = 0;

StbM_SetGlobalTime(0, &timeStamp, &usrData);
```

3.1.2 Configuration of EthTSyn for a Master ECU

Unlike most of the AUTOSAR-based communication protocols, EthTSyn is frame based and not PDU based. Starting with the General Configuration, Figure 3-5 illustrates an example setup.



Short Name: EthTSynGeneral

Dest Phy Addr: 01:80:C2:00:00:0E

Dev Error Detect: ☒

Enable Switch Mgmt: ☐

Eth If Frame Type: /ActiveEcuC/EthIf/EthIfConfigSet/EthTSyn_Frame

Global Time Rx To Uplink Switch Residence Time [ns]:

Global Time Uplink To Tx Switch Residence Time [ns]:

Hardware Timestamp Support: ☒

Main Function Period [s]: 0.005

Master Slave Conflict Detection: ☐

Message Compliance: ☐

Safe Bsw Checks: ☐

Switch Mgmt Gm Capable: ☐

Switch Mgmt Main Function Period [s]:

Switch Mgmt Rx Message Buffer Count:

Switch Mgmt Rx Message Buffer Payload Size [Byte]:

Switch Mgmt Switch Timestamp Support: ☐

Switch Mgmt Tx Mgmt Obj Queue Size:

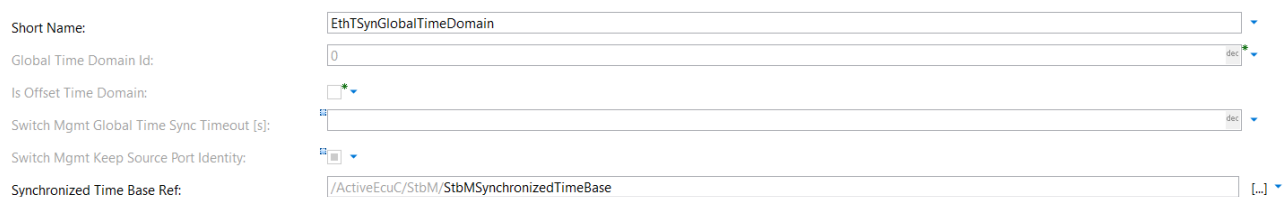
User Config File:

Version Info Api: ☐

Figure 3-5 Configuration of EthTSynGeneral for a Master ECU

- > **Dest Phy Addr:** should be adapted to what is specified but still it is configurable as it is a logical agreement.
- > **EthIf Frame Type:** should reference the configured EthIfFrameOwnerConfig for the specified Ethernet Type (0x88F7).
- > **Hardware Timestamp Support:** If the underlying Ethernet stack supports hardware timestamping, this is set as checked. Otherwise, handling of the ingress and egress timestamps will be offloaded to StbM. The latter case will reduce the accuracy (please refer to section 2).

Now we need to configure a Global Time Domain for our system. To do so, we create one container of type EthTSynGlobalTimeDomain¹ and set the parameters as shown in Figure 3-6.



Short Name: EthTSynGlobalTimeDomain

Global Time Domain Id: 0

Is Offset Time Domain: ☐

Switch Mgmt Global Time Sync Timeout [s]:

Switch Mgmt Keep Source Port Identity: ☐

Synchronized Time Base Ref: /ActiveEcuC/StbM/StbMSynchronizedTimeBase

Figure 3-6 Configuration of EthTSynGlobalTimeDomain

- > **Global Time Domain Id:** this is the global time domain ID and is automatically derived from the StbMSynchronizedTimeBase referenced by EthTSynSynchronizedTimeBaseRef. This value is something which will be appeared in domainNumber field of PTP common Header.

¹ /MICROSAR/EthTSyn/EthTSynGlobalTimeDomain

- > **Synchronized Time-Base Ref:** this parameter shall reference the corresponding StbM Time-Base Reference which was created in the previous sub-clause.

In the next step, we need to configure one EthTSyn-Port1. This port is virtually an interface to the bus (Figure 3-7).

Short Name:	EthTSynPortConfig
Frame Prio:	dec
Global Time Debounce Time [s]:	0.05
Global Time Eth If Ref:	/ActiveEcuC/EthIf/EthIfConfigSet/Connector_ECU_8350b8a97fdb67b6_b9019392
Port Number:	0
Switch Management Eth Switch Port Ref:	

Figure 3-7 Configuration of an EthTSyn Port

In our example setup, we have assumed that the ethernet communication is not over virtual networks². Otherwise, the PCP of the V-LAN tags can be configured via Frame Prio.

- > **Global Time Debounce Time:** This represents the configuration of a TX debounce time for Sync and Follow_Up messages compared to a message before with the same PDU.
- > **Global Time Eth If Ref:** This represents the reference to the Ethernet interface taken to communicate the PTP messages.

Since we are configuring a Master node, we should set the created port's role to Global Time Master and configure its properties as follows:

Short Name:	EthTSynGlobalTimeMaster
Cyclic Msg Resume Time [s]:	0
Follow Up Sent Call Back Function:	
Global Time Tx Crc Secured:	CRC_NOT_SUPPORTED
Global Time Tx Period [s]:	0.125
Immediate Time Sync:	☐
Is System Wide Global Time Master:	☒
Sync Sent Call Back Function:	
TLV Follow Up OFS Sub TLV:	☐
TLV Follow Up Status Sub TLV:	☐
TLV Follow Up Time Sub TLV:	☐
TLV Follow Up User Data Sub TLV:	☐
Transmit Announce:	☐

Figure 3-8 Configuration of an EthTSyn Master Port

- > **Global Time Tx Period:** This parameter defines the cycle of PTP Sync/Follow_Up messages.
- > **Is System Wide Global Time Master:** This determines whether this Port is the GTM or just a Master port.

¹ /MICROSAR/EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig

² The ethernet frames are not VLAN-tagged

As the final step, we need to add the propagation delay related configuration. For this, one container of type EthTSynPdelayConfig shall be created and configured accordingly¹.



Short Name:	EthTSynPdelayConfig
Always AsCapable:	☒
Global Time Pdelay Resp Enable:	☒
Global Time Propagation Delay [s]:	0
Global Time Tx Pdelay Req Period [s]:	0
Pdelay Average Weight:	1
Pdelay Latency Threshold [s]:	1
Pdelay Req Allowed Lost Responses:	1
Pdelay Resp And Resp Follow Up Timeout [s]:	0
Use Static Pdelay:	☐

Figure 3-9 Configuration of Propagation Delay for a Master Port

- > **Always As Capable:** This parameter sets AsCapable to true without checking for a gPTP-capable counterpart via Pdelay-Mechanism. The AsCapable flag will not be influenced by the Pdelay-Mechanism at all. According to IEEE 802.1AS this parameter must be false at start-up and is set automatically to true if a gPTP-capable device at the other end of the link is detected.
- > **Global Time Pdelay Resp Enable:** This parameter enables the EthTSyn port to participate in the propagation delay calculation.

These are the main configuration parameters of StbM and EthTSyn modules required for the basic functionality of Time Synchronization over Ethernet acting as the GTM. Even though the role of nodes are mutually exclusive, some of these parameters consider the properties which is in common between different use cases and roles. In the following sub-section, the configuration of our second example scenario is to be shown.

3.2 Use case 2 - ECU Acting as a Slave ECU

In this scenario, the configuration of StbM and EthTSyn shall be adapted to realize the Slave functionality. The Slave ECU will receive Sync/Follow_Up PTP messages from the bus and trigger the propagation delay requests every second (Figure 3-10).

¹ /MICROSAR/EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSynPdelayConfig

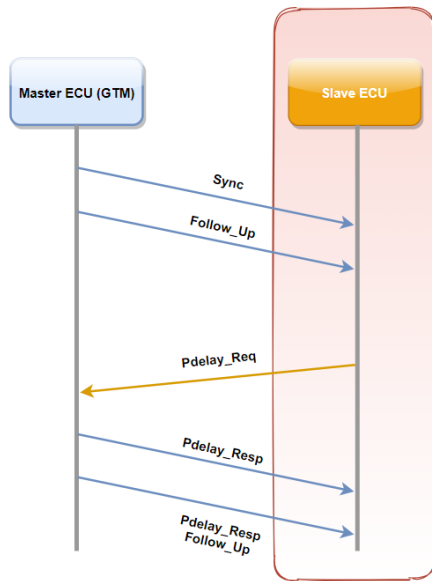


Figure 3-10 Example Configuration 2 - ECU as a Slave Node

The configuration of a Slave ECU is very similar to of Master's except the role related parameters. Due to this, only the parameters required to change are shown in the following:

3.2.1 Configuration of StbM for a Slave ECU

In the configuration of the Synchronized Time-Base which was discussed previously¹, the parameter Is System Wide Global Time Master should be unchecked (Figure 3-11). When doing so, the role of this Synchronized Time-Base will change to Slave. Additionally, there is no need to call `StbM_SetGlobalTime()` anymore as this is the job of a Master node.

Short Name:	StbMSynchronizedTimeBase
Allow Master Rate Correction:	☒
Allow System Wide Global Time Master:	☒
Clear Timeleap Count:	dec
Is System Wide Global Time Master:	☐
Master Rate Deviation Max:	dec
Offset Time Base:	...
Status Notification Callback:	dec
Status Notification Mask:	dec
Store Timebase Non Volatile:	dec
Sync Loss Timeout [ms]:	dec
Synchronized Time Base Identifier:	0
Time Leap Future Threshold [ms]:	dec
Time Leap Past Threshold [ms]:	dec

Figure 3-11 Configuration of Synchronized Time-Base Container for a Slave ECU

This is the only parameter which should change the roles and the rest of configuration remains the same as configuring StbM for a Master node. Please note that the focus of the configuration is to bring the basic functionality up and not to go through the optional setups e.g. configuring Time Correction, Record Blocks, etc.

¹ /MICROSAR/StbM/StbMSynchronizedTimeBase/StbMIsSystemWideGlobalTimeMaster

3.2.2 Configuration of EthTSyn for a Slave ECU

In the configuration of Global Time Domain and Port Config, Port Role¹ should change to Global Time Slave in the first step. This causes not transmitting Sync/Follow_Up messages anymore compared to a Master port. Additionally, in the configuration of propagation delay², the following changes shall apply:

Short Name: EthTSynPdelayConfig

Always AsCapable: ☒

Global Time Pdelay Resp Enable: ☐

Global Time Propagation Delay [s]: 0

Global Time Tx Pdelay Req Period [s]: 1

Pdelay Average Weight: 0

Pdelay Latency Threshold [s]: 0

Pdelay Req Allowed Lost Responses: 0

Pdelay Resp And Resp Follow Up Timeout [s]: 0

Use Static Pdelay: ☐

Figure 3-12 Configuration of Propagation Delay for a Slave Node

- > **Global Time Pdelay Resp Enable:** This parameter can be disabled as a Slave port essentially does not expect to receive Pdelay requests in conventional setups.
- > **Global Time Tx Pdelay Req Period:** This defines the frequency of Pdelay requests triggered by a Slave node. As of our example scenario, every 1s would be an acceptable choice.

And the configuration of a Slave ECU is now done.

3.3 Use case 3 - Bridged network - ECU Acting as the Mgmt. CPU

In these types of setups, there are typically more time users which are connected to a switch forming an ethernet LAN. For every Global Time Domain, there is only one GTM. Even though the role of GTM can be delegated to other nodes of the network, this only happens in case of temporary down time of the GTM and will be taken over as soon as the GTM is functionally up.

Having a single GTM does not mean that there will only be one Master node in the network as GTM/Slaves might be reachable via one or more network hops. No matter how the network connectivity looks like as long as the nodes are within a single multicast domain and physically connected, they can see each other and should be able to communicate. Therefore, since every network hop implies a forwarding entity, even having only one GTM will result in multiple Master nodes in the network.

After all, the role of a Master node is correlated to the definition of ports in EthTSyn module. This means that, having such a scenario, we might need to configure a mixed setup depending on the network connectivity. The example network considered for our sample configuration can be illustrated via Figure 3-13.

¹ /MICROSAR/EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSynPortRole

² /MICROSAR/EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSynPdelayConfig

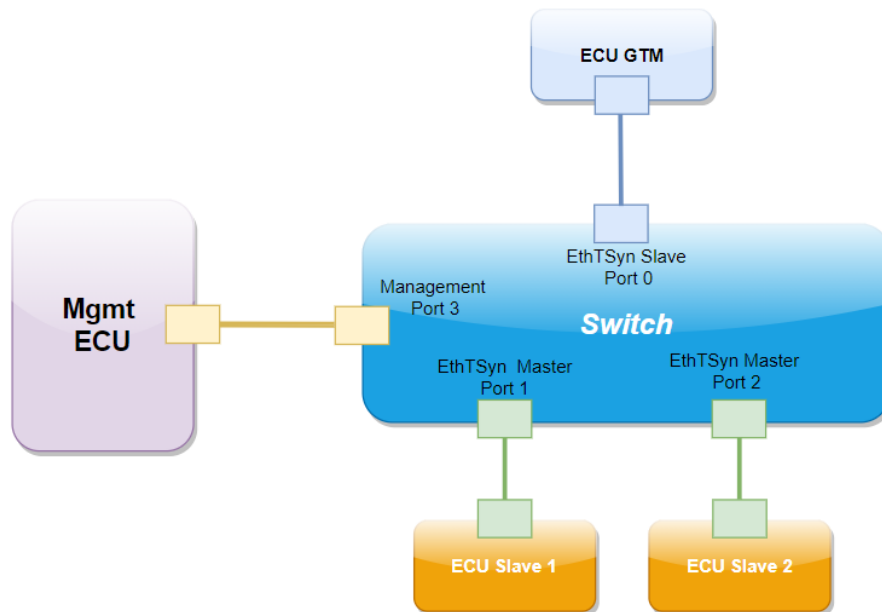


Figure 3-13 Example Time-aware Network

In the figure above, there are two Slave ECUs connected directly to the switch, the GTM ECU, and the ECU running the EthTSyn and managing the PTP communication over switch (Mgmt. CPU). The Mgmt. CPU is considered as a Slave ECU from application point of view (also from StbM perspective) but it also acts as the forwarding node (Master node) from EthTSyn point of view.

In this setup, the GTM will transmit Sync/Follow_Up messages to the network (multicast them) and the Mgmt. CPU should trap these PTP messages, measure the incurred delays by the switch, adapt the correctionField of the PTP Follow_Up frames and forward them to the remaining of the network. Therefore, Sync/Follow_Up messages will reach to the connected Slave ECUs via two hops. In contrast, propagation delay is considered as link to link and there is no concept of forwarding here.

3.3.1 Configuration of StbM for the Mgmt. CPU

The configuration of StbM remains the same as demonstrated earlier in Configuration of StbM for a Slave ECU.

3.3.2 Configuration of EthTSyn for the Mgmt. CPU

In the following section, an example configuration for our scenario is demonstrated as well as a short description of the configuration parameters. Kindly note that this is only one possible configuration and might be slightly different for different requirements. In the first step, we go through the general configuration¹.

¹ /MICROSAR/EthTSyn/EthTSynGeneral

3.3.2.1 General Configuration

Short Name:	EthTSynGeneral
Dest Phy Addr:	01:80:C2:00:00:0E
Dev Error Detect:	☒
Enable Switch Mgmt:	☒
Eth If Frame Type:	/ActiveEcuC/EthIf/EthIfConfigSet/EthTSyn_EthTSyn
Global Time Rx To Uplink Switch Residence Time [ns]:	2
Global Time Uplink To Tx Switch Residence Time [ns]:	2
Hardware Timestamp Support:	☒
Main Function Period [s]:	0.005
Master Slave Conflict Detection:	☐
Message Compliance:	☐
Safe Bsw Checks:	☐
Switch Mgmt Bridge Mode:	TRANSPARENT_CLOCK
Switch Mgmt Gm Capable:	☒
Switch Mgmt Main Function Period [s]:	0.005
Switch Mgmt Rx Message Buffer Count:	6
Switch Mgmt Rx Message Buffer Payload Size [Byte]:	42
Switch Mgmt Switch And Host As One Time Aware System:	☐
Switch Mgmt Switch Timestamp Support:	☒
Switch Mgmt Tx Mgmt Obj Queue Size:	5
User Config File:	
Version Info Api:	☐

Figure 3-14 General Configuration of EthTSyn for the Mgmt. CPU

- > **Dest Phy Addr:** As discussed earlier, this address is used for the destination MAC address of outgoing Sync, Follow_Up, Pdelay_Req, Pdelay_Resp, and Pdelay_Resp_Follow_Up messages.
- > **Enable Switch Mgmt.:** When configuring the ECU in management mode, this parameter shall be enabled.
- > **Eth If Frame Type:** As mentioned earlier, the chosen frame owner determines which frames, with respect to their ethertype, are received. The reference shall be set to /MICROSAR/EthIf/EthIfConfigSet/EthIfFrameOwnerConfig with the frame type of 0x88F7.
- > **Global Time Rx to Uplink Switch Residence Time:** This parameter sets the default value used for the Rx residence time of the Ethernet Switch [Ingress to Uplink]. This value is used by the EthTSyn if the calculation of the residence time failed. This parameter refers to the time from T4 to T5 in Figure 2-19.
- > **Global Time Up link to Tx Switch Residence Time:** This parameter sets the default value used for the Tx residence time of the Ethernet Switch [Uplink to Egress]. This value is used by the EthTSyn if the calculation of the residence time failed. This parameter refers to the time from T6 to T7 in Figure 2-19.
- > **Hardware Timestamp Support:** Activates/Deactivates the hardware time stamping functionality of the Ethernet hardware. If the hardware supports timestamping, this parameter can be set to TRUE. Otherwise, the timestamping will be handled by StbM.
- > **Master/Slave Conflict Detection:** This option enables conflict detection of Master/Slave nodes in the bus. When a master node receives a sync message, it reports a runtime

error and when a Slave node receives a sync message with a different sourcePortIdentity, it reports a runtime error.

- > **Message Compliance:** This parameter shall be set according to the requirements. It should be enabled if IEEE 802.1AS compliant message format will be used. Otherwise, IEEE 802.1AS message format with AUTOSAR extension will be used.
- > **Switch Mgmt. Bridge Mode:** To be used for choosing the bridge mode of the Mgmt. CPU. For further information about the difference of these modes, please refer to IEEE1588 Chapters 9 and 10. For our example use case, Transparent Clock the right choice as we intend to forward the Synchronization messages to the Slave nodes and influence the switch residence time via correctionField [5].
- > **Switch Mgmt. Gm Capable:** When this parameter is set, in case of a Sync-Timeout being detected on a slave port (no Sync/Follow_Up is received from the GTM), the management node (ECU) will take over the GTM role. This happens in case of GTM goes down due to some failure and once it is up and running, the role is taken back.
- > **Switch Mgmt. Rx Message Buffer Count:** This parameter is to determine the number of required Rx message buffers in bridge mode. It depends on the port configuration and how many Rx messages can be pending at a point in time and waiting to be processed. E.g. When only one slave and one master ports are configured, from the master port, maximum one Pdelay_Req can be pending, and from the slave port, one Sync, one Follow_Up, one Pdelay_Resp, and one Pdelay_Resp_Follow_Up can be waiting for processing in the worst case. Therefore, in total, 5 would be a suitable number.
- > **Switch Mgmt. Rx Message Buffer Payload Size:** This parameter shall be configured such that the message buffer can store the received PTP frames. The Common PTP Header (34 Byte) is excluded from this size here.
- > **Switch Mgmt. Switch Timestamp Support:** When EthTSyn is used to manage the Time Sync Protocol on an Ethernet Switch, this parameter is used to enable the support of Timestamps generated by the Ethernet Switch. For EthTSyn-Ports mapping to an Ethernet Switch Port, the Timestamps provided by the Ethernet Switch will be used. For EthTSyn-Ports mapping to an Ethernet Controller but not to an Ethernet Switch Port (i.e. End-Station Port), the configured Timestamp Method will be used.
- > **Switch Mgmt. Tx Mgmt. Obj Queue Size:** The same analogy as Rx Message Buffer Count but in Tx direction. Please note that this parameter refers to the number of Tx messages which can be pending waiting for their Egress timestamp to be available (Tx event messages).

In the next step, we will configure the Global Time Domain for our scenario. It is worth noting that EthTSyn module can administrate several global time domains while, in itself, form a hierarchy of domains and sub-domains. If the EthTSyn exists, it is assumed that at least one global time domain exists.

3.3.2.2 Configuration of a Global Time Domain

The following figure shows the configuration of the Time Domain for our example use-case¹.

¹ /MICROSAR/EthTSyn/EthTSynGlobalTimeDomain

Short Name:	EthTSynGlobalTimeDomain
Global Time Domain Id:	0
Is Offset Time Domain:	☐
Switch Mgmt Global Time Sync Timeout [s]:	0.375
Switch Mgmt Keep Source Port Identity:	☐
Synchronized Time Base Ref:	/ActiveEcuC/StbM/StbMSynchronizedTimeBase

Figure 3-15 The configuration of Global Time Domain for the Mgmt. CPU

- > **Global Time Domain Id:** This is the value of global time domain ID. Note: This value is automatically derived from the StbM Synchronized Time-Base referenced by EthTSyn Synchronized Time-Base Ref.
- > **Is Offset Time Domain:** Information only (computed automatically). This parameter is set to 'TRUE' if the Time Domain is an offset Time Domain (Time Domain Id [16;31]).
- > **Switch Mgmt. Global TimeSync Timeout:** When Switch Mgmt. Gm Capable is enabled, this parameter sets the timeout for which the Mgmt. CPU is waiting for the reception of a new Sync message on its slave port¹ before a Grand Master timeout is expired and the Bridge begins to generate its own Sync and Follow_Up messages.
- > **Switch Mgmt. Keep Source Port Identity:** When this parameter is enabled, the source port identity of forwarded Sync/Follow_Up messages is not modified.
- > **Synchronized Time-Base Ref:** The reference to the relevant synchronized Time-Base.

The configuration of the Time Domain is done and now we need to configure the required EthTSyn ports. Referring to Figure 3-11, at first, we need to know how many ports are required. As earlier mentioned, each EthTSyn port resembles an interface to the bus. Since the switch is managed by the ECU, all involved switch ports are the ECU's ports from EthTSyn perspective. The correlation of EthTSyn, EthIf, and the switch in a bridged network can be seen in the following figure:

¹ GTM is connected to the Switch via a Slave port.

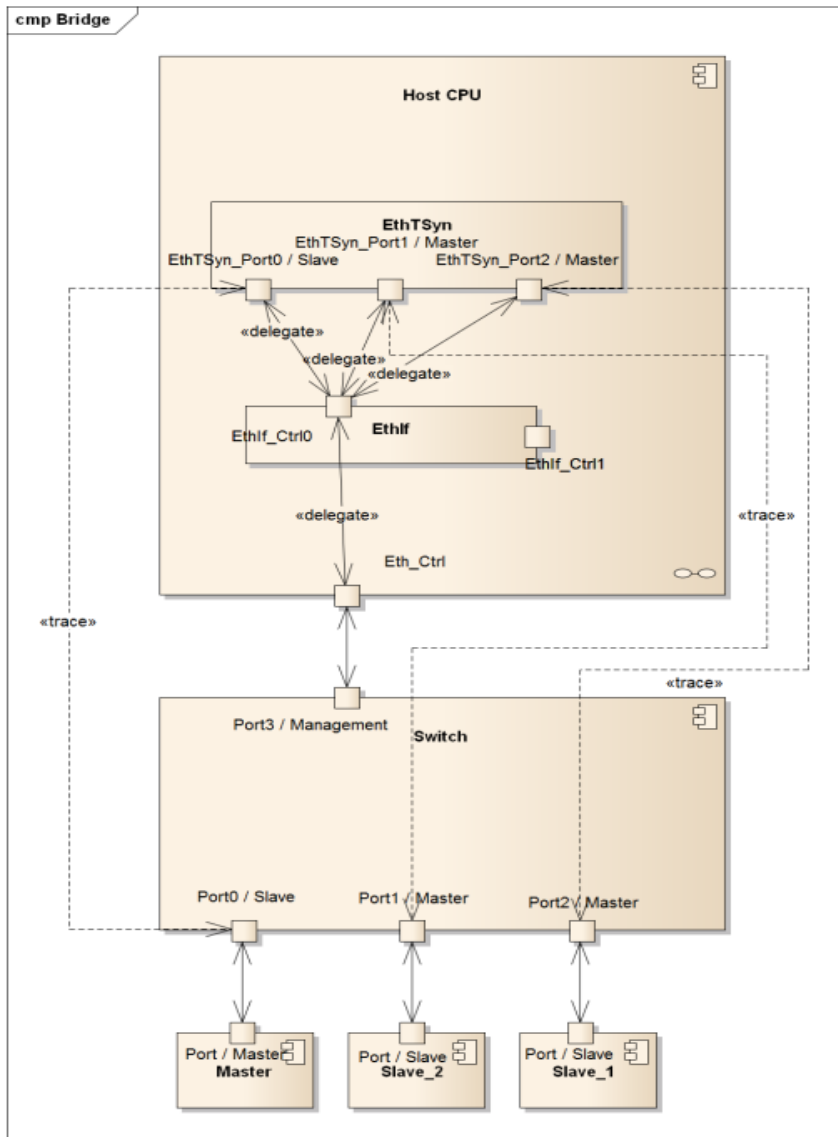


Figure 3-16 Correlation of EthTSyn, EthIf, and the Switch in a Bridged Network [2]

Accordingly, for our example scenario:

- > Slave ECU 1: This should be a Master port as the Mgmt. CPU forwards the received Sync/Follow_Up messages to the Slave nodes.
- > Slave ECU 2: This should also be a Master port for the same reason above.
- > GTM ECU: This should be a Slave port because it connects the GTM to the network and receives Sync/Follow_Up messages.
- > Mgmt. CPU: Even though this node forwards PTP traffic, itself is a time user and receives Sync/Follow_Up messages from the GTM. Therefore, this port also should be a Slave.

3.3.2.3 Configuration of EthTSyn Ports

At this stage, we need to configure 4 EthTSyn ports two of which as Master ports and the other two Slaves. Therefore, we create 4 EthTSynPortConfig¹ and configure them as follows:

Short Name:	EthTSynPortConfig_MasterPort1
Frame Prio:	dec
Global Time Debounce Time [s]:	0.05
Global Time Eth If Ref:	/ActiveEcuC/EthIf/EthIfConfigSet/EthIfController_NON_VLAN
Port Number:	1
Switch Management Eth Switch Port Ref:	/ActiveEcuC/EthSwt/EthSwtConfig/EthSwtPort_MasterPort1

Figure 3-17 Sample Configuration of EthTSynPortConfig Container for Mgmt. CPU

- > **Frame Prio:** This parameter is used to set the priority of the Ethernet Frames when the time synchronization is over a VLAN network. The field encodes the PCP field of the VLAN tag for the outgoing PTP messages.
- > **Global Time Debounce Time:** This represents the configuration of a TX debounce time for Sync and Follow_Up messages compared to a message before with the same PDU.
- > **Global Time Eth If Ref:** This parameter represents the reference to the Ethernet Interface to be used for communicating the PTP frames to/from.
- > **Port number:** Determines the Port Number.
- > **Switch Management Eth Switch Port Ref:** In the bridged mode config, this reference is used to assign the EthTSyn Port to the relevant Switch Port (hardware port).

All four ports can be created in the same way. There is an exception for the Mgmt. CPU port compared to the others. This port is considered as an End-Station and configured irrespective of its connection to the switch (like configuring a Slave ECU). Therefore, it does not have an ethernet Switch Port assignment and this parameter shall be empty as follows:

Short Name:	EthTSynPortConfig_EndStationPort
Frame Prio:	dec
Global Time Debounce Time [s]:	0.05
Global Time Eth If Ref:	/ActiveEcuC/EthIf/EthIfConfigSet/EthIfController_NON_VLAN
Port Number:	0
Switch Management Eth Switch Port Ref:	

Figure 3-18 Configuration of the Management Port

The configuration of EthTSyn port is not yet finished.

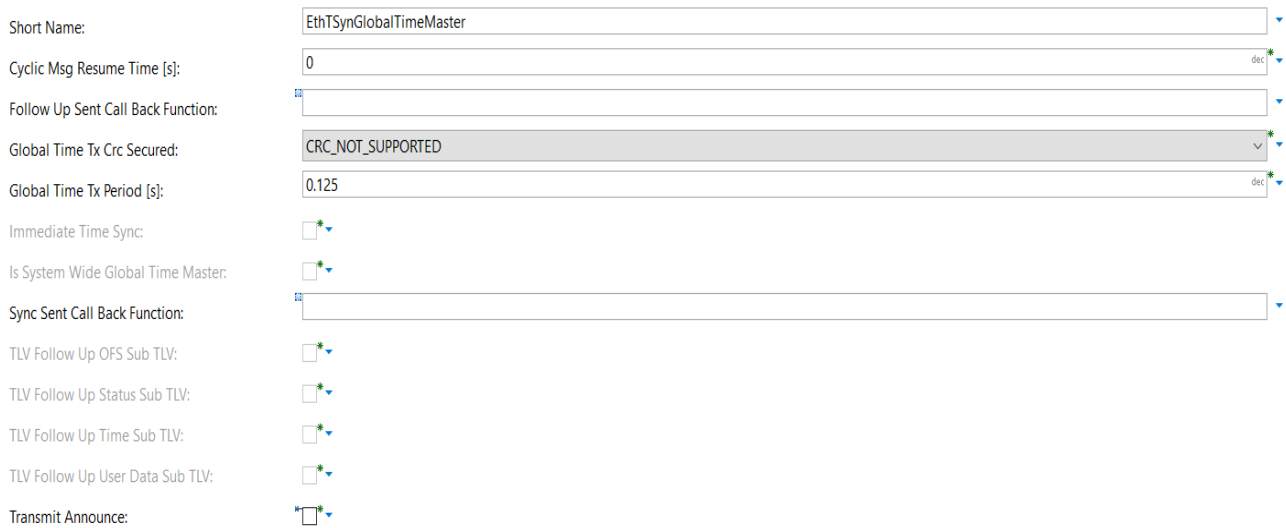
In the next step, the role of the ports shall be chosen via EthTSynPortRole². Depending on the port role, further configuration parameters shall be set according to the requirements.

¹ /MICROSAR/EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig

² /MICROSAR/EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSynPortRole

3.3.2.4 Configuration of Master Ports

For our example, we should have two Master ports for the two Slave nodes connected to the switch. These two ports can be configured as Figure 3-19 shows:



Short Name:	EthTSynGlobalTimeMaster
Cyclic Msg Resume Time [s]:	0
Follow Up Sent Call Back Function:	
Global Time Tx Crc Secured:	CRC_NOT_SUPPORTED
Global Time Tx Period [s]:	0.125
Immediate Time Sync:	☐
Is System Wide Global Time Master:	☐
Sync Sent Call Back Function:	
TLV Follow Up OFS Sub TLV:	☐
TLV Follow Up Status Sub TLV:	☐
TLV Follow Up Time Sub TLV:	☐
TLV Follow Up User Data Sub TLV:	☐
Transmit Announce:	☐

Figure 3-19 Sample Configuration of Global Time Master Ports in Bridged Mode

- > **Cyclic Msg Resume Time:** Once an immediate message is transmitted, this parameter determines the time to start transmission of the cyclic messages.
- > **Follow Up Sent Call Back Function:** This function is called after a FOLLOW_UP message has successfully been transmitted/Forwarded. The call-back function has the following signature:

```
void EthTSyn_Follow_UpSentCallback(EthTSyn_IntPortIdxType PortIdx, const uint8* TxBufPtr);
```

- > **Global Time Tx Crc Secured:** This represents the configuration of whether CRC is supported (for further information about the details of TLV, please refer to [4]).
- > **Global Time Tx Period:** This defines the period for Sync/Follow_Up messages.
- > **Immediate Time Sync:** This parameter enables transmission of immediate Sync messages once there is an update for the Global Time from StbM.
- > **Is System Wide Global Time Master:** This represents whether the Master port represents a GTM.
- > **Sync Sent Call Back Function:** This function is called after a Sync message has successfully been transmitted. The call-back function has following signature:

```
void EthTSyn_SyncSentCallback(EthTSyn_IntPortIdxType PortIdx, const uint8* TxBufPtr );
```

- > **TLV Follow_Up OFS Sub TLV:** This represents the configuration of whether an AUTOSAR Follow_Up TLV OFS Sub-TLV is used or not.
- > **TLV Follow_Up Status Sub TLV:** This represents the configuration of whether an AUTOSAR Follow_Up TLV Status Sub-TLV is used or not.

- > **TLV Follow_Up Time Sub TLV:** This represents the configuration of whether an AUTOSAR Follow_Up TLV Time Sub-TLV is used or not.
- > **TLV Follow_Up UserData Sub TLV:** This represents the configuration of whether an AUTOSAR Follow_Up TLV UserData Sub-TLV is used or not.
- > **Tx Announce:** Enables/Disables the transmission of Announce messages for the Master port.

3.3.2.5 Configuration of Slave Ports

For configuring the Slave ports of our example, we create two `EthTSynPortConfig`¹, set the `EthTSynPortRole`² to `EthTSynGlobalTimeSlave` and adapt the parameters as demonstrated here:



Short Name:	EthTSynGlobalTimeSlave
Announce Max Step Removed:	100
Announce Timeout [s]:	100
Enable Announce:	☐
Enable Source Port Identity Check:	☐
Global Time Follow Up Timeout [s]:	0.125
Rx Crc Validated:	CRC_IGNORED

Figure 3-20 Sample Configuration of the Slave Ports for Mgmt. CPU

- > **Announce Max Step Removed:** This represents the maximum number of network hops between the GTM and this slave. Note: Only used when `EthTSynEnableAnnounce==true`.
- > **Announce Timeout:** Determines the maximum delay time in ms between incoming Announce messages sent by the master. Note: Only used when `Enable Announce==true`.
- > **Enable Announce:** When this option is enabled, the slave has to receive a valid announce message before synchronization is possible. The slave stores the `ClockIdentity` of the `GrandMaster` (received via announce message) and validates `Sync/Follow_Up` messages using this clock identity.
- > **Enable Source Port Identity Check:** This parameter specifies whether the `SourcePortIdentity` of received `Sync/Follow_Up` messages should be verified or not. Note: When enabled, the valid source port identity for `Sync/Follow_Up` messages has to be configured using the container `EthTSynSourcePortIdentityCheckConfig`.
- > **Global Time Follow Up Timeout:** Timeout value of the `Follow_Up` message (of the subsequent `Sync` message). A value of 0 deactivates this timeout observation.
- > **Rx Crc Validated:** Definition of whether validation of the CRC is supported.
 - > **CRC_IGNORED:** `EthTSyn` accepts all `Follow_Up` messages. Possibly included CRCs in any `ArSubTlv` will be ignored.

¹ /MICROSAR/EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig

² /MICROSAR/EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSynPortRole

- > **CRC_NOT_VALIDATED:** EthTSyn only accepts Follow_Up messages where no CRC is included in optional ArSubTlvs (i.e. SubTlv Types: 0x51, 0x61 and 0x34).
- > **CRC_OPTIONAL:** EthTSyn accepts all Follow_Up messages. If an ArSubTlv includes a CRC, the CRC is validated.
- > **CRC_VALIDATED:** EthTSyn only accepts Follow_Up messages where CRC is included in optional ArSubTlvs (i.e. SubTlv Types: 0x28, 0x50, 0x60 and 0x44). The Crc is validated.

Configuration of Master/Slave ports are complete now. Finally, we configure propagation delay for these 4 ports.

3.3.2.6 Configuration of Pdelay for Master/Slave Ports

For each port, we create one EthTSynPdelayConfig¹ and set the parameters as Figure 3-21 and Figure 3-22 for Master and Slave ports respectively.

Short Name: EthTSynPdelayConfig

Always AsCapable: ☒

Global Time Pdelay Resp Enable: ☒

Global Time Propagation Delay [s]: 0

Global Time Tx Pdelay Req Period [s]: 0

Pdelay Average Weight: 1

Pdelay Latency Threshold [s]: 1

Pdelay Req Allowed Lost Responses: 1

Pdelay Resp And Resp Follow Up Timeout [s]: 0

Use Static Pdelay: ☐

Figure 3-21 Sample Configuration of Pdelay for a Master Port

Short Name: EthTSynPdelayConfig

Always AsCapable: ☐

Global Time Pdelay Resp Enable: ☐

Global Time Propagation Delay [ns]: 3500

Global Time Tx Pdelay Req Period [s]: 1

Pdelay Average Weight: 1

Pdelay Latency Threshold [s]: 1

Pdelay Req Allowed Lost Responses: 1

Pdelay Resp And Resp Follow Up Timeout [s]: 0

Use Static Pdelay: ☐

Figure 3-22 Sample Configuration of Pdelay for a Slave Port

- > **Always As Capable:** When enabled, AsCapable is set to true without checking for a Ptp counterpart via Pdelay-Mechanism. Hence for a master port the transmission of Sync and Follow_Up messages is possible without Pdelay measurement. For a slave port, synchronization is possible without a valid Pdelay.
- > **Global Time Pdelay Resp Enable:** This parameter allows disabling Pdelay_Resp / Pdelay_Resp_Follow_Up transmission, if no Pdelay_Req messages are expected. FALSE: No Pdelay requests expected. Pdelay_Resp / Pdelay_Resp_Follow_Up

¹ /MICROSAR/EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSynPdelayConfig

transmission is disabled. TRUE: Pdelay requests expected. Pdelay_Resp / Pdelay_Resp_Follow_Up transmission is enabled.

- > **Global Time Propagation Delay:**
 - > If cyclic propagation delay measurement is enabled, this parameter represents the default value of the propagation delay until the first actually measured propagation delay is available.
 - > If cyclic propagation delay measurement is disabled, this parameter represents the static value of the propagation delay.
 - > If cyclic propagation delay measurement is disabled, this parameter replaces a measured propagation delay by a fixed value.
- > **Global Time Tx Pdelay Req Period:** This represents configuration of the TX period for Pdelay_Req messages. A value of 0 disables the cyclic Pdelay measurement.
- > **Pdelay Average Weight:** To determine the Pdelay, a weighted average calculation can be used instead of always take the last measured value. This parameter represents the weight factor (w) for PathDelay calculation: $\text{NewPathDelay} = (((w - 1) * \text{OldPathDelay}) + \text{CurrentPathDelay}) / w$. Greater values of w causes old value to contribute more than the lastly calculated path delay.
- > **Pdelay Latency Threshold:** Threshold for calculated Pdelay. If a measured Pdelay exceeds EthTSynPdelayLatencyThreshold, this value is discarded, and lost response counter is incremented.
- > **Pdelay Req Allowed Lost Responses:** Determines the number of allowed lost responses of a PDelay_Request. If this count is exceeded, the AsCapable Flag of the Port is set to false.
- > **Pdelay Resp And Resp Follow_Up Timeout:** Timeout value for Pdelay_Resp and Pdelay_Resp_Follow_Up after a Pdelay_Req has been transmitted resp. a Pdelay_Resp has been received. A value of 0 deactivates this timeout observation.
- > **Use Static Pdelay:** When this parameter is set to TRUE, the configured Pdelay value (EthTSynGlobalTimePropagationDelay) is always used as Pdelay. The optional Pdelay measurement does not affect this value.

4 Example Diagram

In this section, a perceptible example is shown to demonstrate how EthTSyn and StbM collaborate in GTM and Slave ECUs for distribution and provision of the Global Time. In order to ease the insight of the diagram, the important time spots are magnified following up.

In this example, as shown in the Figure below, The GTM and the Slave nodes are located on the same timeline and aligned with comparable events.

Assumptions:

- > GTM uses an Os System Timer as StbMLocalTimeHardware starting at 0 ticks
- > Slave ECU uses a GPT Timer as StbMLocalTimeHardware starting at 15 ticks
- > Each tick is 1ms
- > Time diagram starts at 0
- > Sync/Follow_Up messages are transmitted every 125ms
- > Sync/Follow_Up messages reach to the Slave ECU with 20ms of delay
- > Virtual Local Time is demonstrated in ms and shown as TV
- > Transmission of Sync/Follow_Up messages are shown identical but in reality, they are transmitted separately.
- > There is no rate deviation between the Os System Counter in the GTM and the GPT in the Slave ECU ($r = 1$).

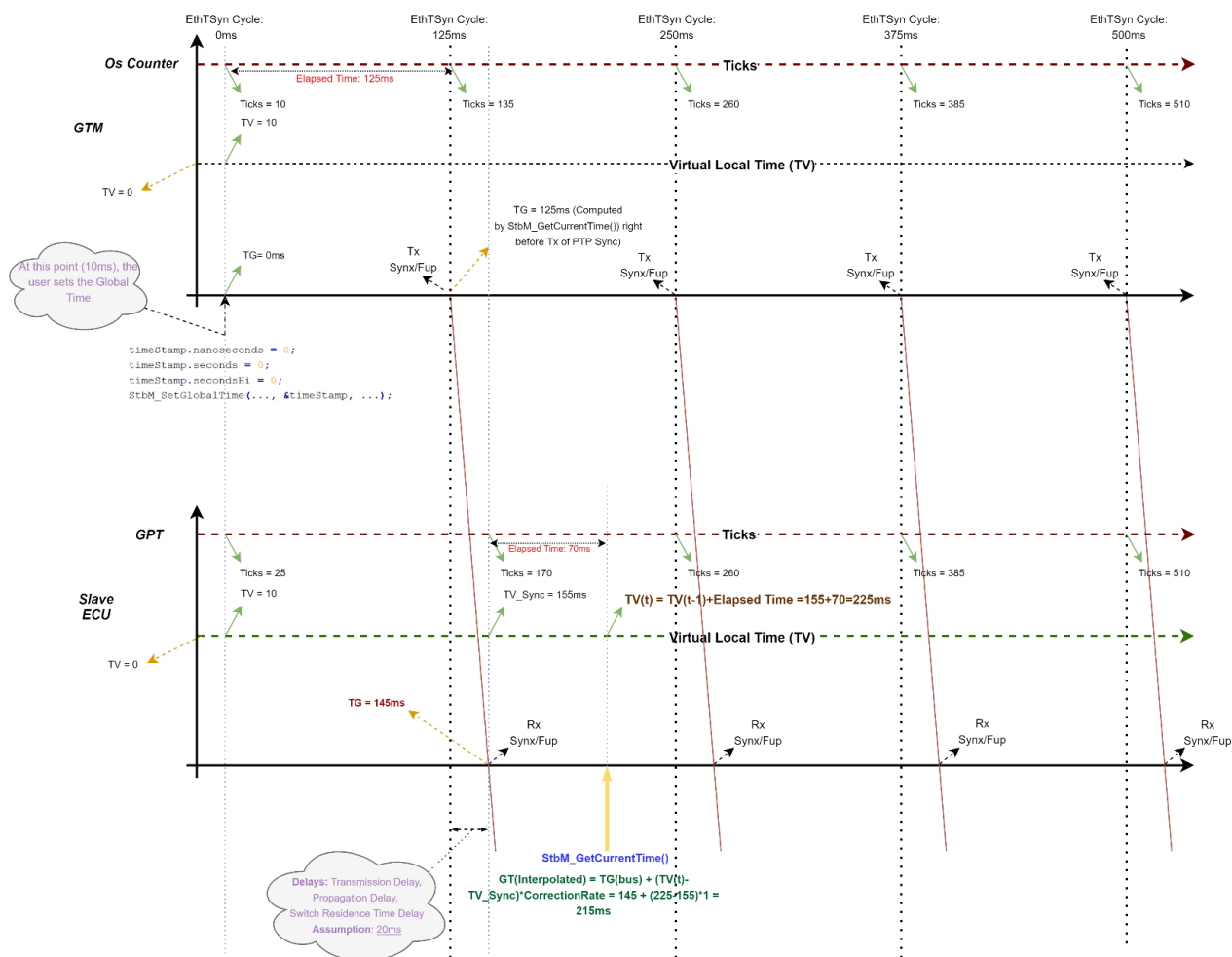


Figure 4-1 Collaboration of EthTSyn and StbM for Distribution and Provision of the Global Time

4.1 Snapshot 1

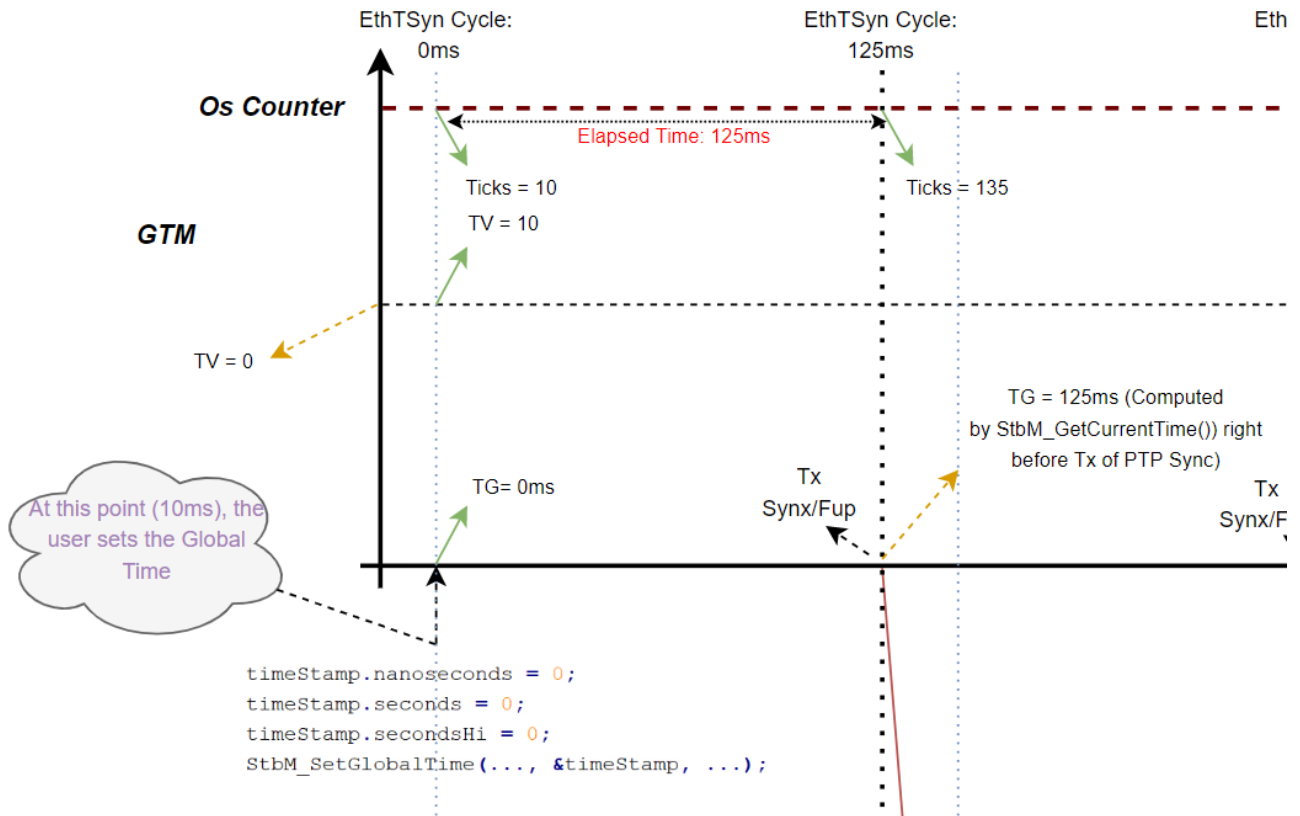


Figure 4-2 Snapshot of StbM and EthTSyn Collaboration in the GTM

As assumed, after 10ms of execution, the user sets the global time to 0 in the GTM via `StbM_SetGlobalTime(...)`. At this moment, TV is updated to 10 (seen as TV(t-1) when used later) and stored as also TV_Sync. After 125ms, which is the configured cycle time of Sync messages, the Sync/Fup are transmitted. The Global Time is captured when transmitting the Sync message through getting the current time (TG+ (TV(t)-TV_Sync)), in which, TV(t) is the Elapsed Time + TV(t-1). Therefore, 125ms is stored to be carried via the Follow_Up message.

4.2 Snapshot 2

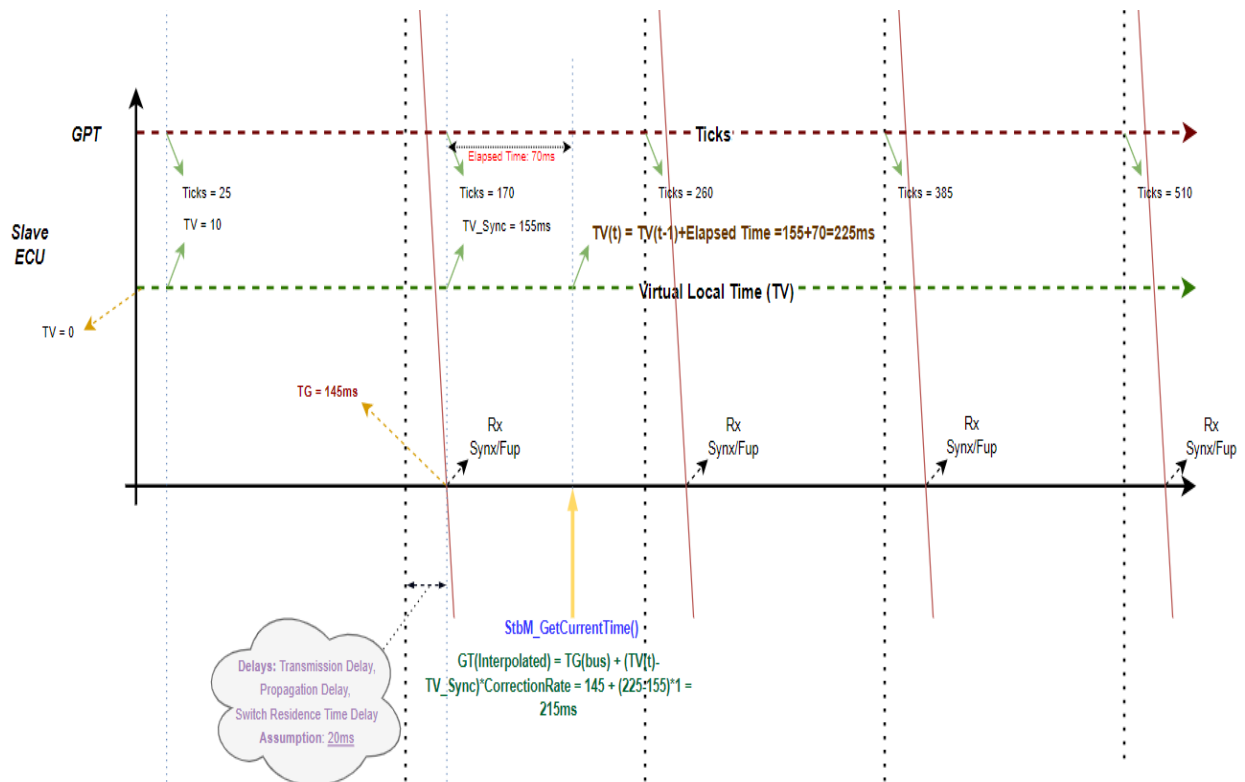


Figure 4-3 Snapshot of StbM and EthTSyn Collaboration in the Slave Node

In the slave side, once the Follow_Up message is received, the Global Time is extracted. The global Time is stored as 145ms (TG) because it is assumed that there are multiple influencing delays. These delays are the transmission delay in the GTM, the residence time of any existing switches in the way, and the propagation delay of the last link.

At the point the Follow_Up message is received; the Virtual Time is updated as TV (in the formula $TV(t-1)$), and additionally stored as TV_Sync. Later, there is a call for StbM_GetCurrentTime(). When this happens, the current value of the GPT tick is read to update the Virtual Time ($TV(t) = \text{Elapsed Time} + TV(t-1)$). This updated Virtual Time is subtracted to the TV_Sync to measure how much time it has passed from the moment latest value of the Global Time was received. Add the value of the Global Time to the measured value, we will have our interpolated Global Time (GT (interpolated)).

Please note that when we subtract TV from TV_Sync, since this demonstrates independent progression of time in the Slave node, any existing rate deviation compared to the clock of the GTM shall apply here to our elapsed time.

5 Glossary and Abbreviations

Term	Description
BMCA	Best Master Clock Algorithm
CRC	Cyclic Redundancy Checksum
Debounce Time	Minimum gap between two Tx messages with the same PDU.
EthTSyn	Time Synchronization Provider module for Ethernet
Follow_Up	Time transport message (Follow-Up)
GTM	Global Time Master
StbM	Synchronized Time_Base Manager
OFS	Offset synchronization
Pdelay	Propagation / path delay as given in IEEE 802.1AS
Pdelay_Req	Propagation / path delay request message
Pdelay_Resp	Propagation / path delay response message
Pdelay_Resp_Follow_Up	Propagation / path delay Follow-Up message
PDU	Protocol Data Unit
PTP	Precision Time Protocol
SoAd	Socket Adapter
Sync	Time synchronization message (Sync)
TLV	Type, Length, Value field (acc. to IEEE 802.1AS)
VLAN	Virtual Local Area Network

6 Contact

Visit our website for more information on

- > News
- > Products
- > Demo software
- > Support
- > Training data
- > Addresses

www.vector.com